

COMPLETE TECHNICAL INTERVIEW PREPARATION GUIDE

Contents

Complete Technical Interview Preparation Guide	4
Table of Contents	4
1. Java	5
1.1 What is it?	5
1.2 When is it used?	5
1.3 How is it used? (Typical workflow)	5
1.4 Why is it used?	5
1.5 Core Syntax & Examples	6
1.6 Basic Interview Questions	8
1.7 Advanced Interview Questions	8
2. Python	10
2.1 What is it?	10
2.2 When is it used?	10
2.3 How is it used?	10
2.4 Why is it used?	10
2.5 Core Syntax & Examples	10
2.6 Basic Interview Questions	12
2.7 Advanced Interview Questions	13
3. C++	15
3.1 What is it?	15
3.2 When is it used?	15
3.3 How is it used?	15
3.4 Why is it used?	15
3.5 Core Syntax & Examples	15
3.6 Basic Interview Questions	17
3.7 Advanced Interview Questions	18
4. JavaScript	20
4.1 What is it?	20
4.2 When is it used?	20
4.3 How is it used?	20
4.4 Why is it used?	20
4.5 Core Syntax & Examples	20
4.6 Basic Interview Questions	22
4.7 Advanced Interview Questions	23
5. HTML	25

5.1 What is it?	25
5.2 When is it used?	25
5.3 How is it used?	25
5.4 Why is it used?	25
5.5 Core Syntax & Examples	25
5.6 Basic Interview Questions	27
5.7 Advanced Interview Questions	27
6. CSS	29
6.1 What is it?	29
6.2 When is it used?	29
6.3 How is it used?	29
6.4 Why is it used?	29
6.5 Core Syntax & Examples	29
6.6 Basic Interview Questions	31
6.7 Advanced Interview Questions	32
7. React	33
7.1 What is it?	33
7.2 When is it used?	33
7.3 How is it used?	33
7.4 Why is it used?	33
7.5 Core Syntax & Examples	33
7.6 Basic Interview Questions	34
7.7 Advanced Interview Questions	35
8. Angular	36
8.1 What is it?	36
8.2 When is it used?	36
8.3 How is it used?	36
8.4 Why is it used?	36
8.5 Core Syntax & Examples	36
8.6 Basic Interview Questions	37
8.7 Advanced Interview Questions	38
9. Streamlit	39
9.1 What is it?	39
9.2 When is it used?	39
9.3 How is it used?	39
9.4 Why is it used?	39
9.5 Core Syntax & Examples	39
9.6 Basic Interview Questions	40
9.7 Advanced Interview Questions	40
10. Scikit-learn	42
10.1 What is it?	42
10.2 When is it used?	42
10.3 How is it used?	42
10.4 Why is it used?	42
10.5 Core Syntax & Examples	42
10.6 Basic Interview Questions	43
10.7 Advanced Interview Questions	44
11. TensorFlow.js	46
11.1 What is it?	46

11.2 When is it used?	46
11.3 How is it used?	46
11.4 Why is it used?	46
11.5 Core Syntax & Examples	46
11.6 Basic Interview Questions	47
11.7 Advanced Interview Questions	48
12. MySQL	49
12.1 What is it?	49
12.2 When is it used?	49
12.3 How is it used?	49
12.4 Why is it used?	49
12.5 Core Syntax & Examples	49
12.6 Basic Interview Questions	50
12.7 Advanced Interview Questions	51
13. Oracle Database	53
13.1 What is it?	53
13.2 When is it used?	53
13.3 How is it used?	53
13.4 Why is it used?	53
13.5 Core Syntax & Examples	53
13.6 Basic Interview Questions	55
13.7 Advanced Interview Questions	55
14. MongoDB	57
14.1 What is it?	57
14.2 When is it used?	57
14.3 How is it used?	57
14.4 Why is it used?	57
14.5 Core Syntax & Examples	57
14.6 Basic Interview Questions	58
14.7 Advanced Interview Questions	59
15. Git	61
15.1 What is it?	61
15.2 When is it used?	61
15.3 How is it used?	61
15.4 Why is it used?	61
15.5 Core Syntax & Examples	61
15.6 Basic Interview Questions	62
15.7 Advanced Interview Questions	63
16. VS Code	65
16.1 What is it?	65
16.2 When is it used?	65
16.3 How is it used?	65
16.4 Why is it used?	65
16.5 Key Features & Usage Examples	65
16.6 Basic Interview Questions	66
16.7 Advanced Interview Questions	66
17. Eclipse	68
17.1 What is it?	68
17.2 When is it used?	68

17.3 How is it used?	68
17.4 Why is it used?	68
17.5 Key Features & Workflow Examples	69
17.6 Basic Interview Questions	69
17.7 Advanced Interview Questions	69
18. IntelliJ IDEA	71
18.1 What is it?	71
18.2 When is it used?	71
18.3 How is it used?	71
18.4 Why is it used?	71
18.5 Key Features & Examples	72
18.6 Basic Interview Questions	72
18.7 Advanced Interview Questions	72

Complete Technical Interview Preparation Guide

Languages · Web Technologies · AI/ML · Databases · Tools

A comprehensive, section-wise reference covering What it is, When it is used, How it is used, Why it is used, Core Syntax, and Interview Questions (Basic + Advanced) for every skill on your resume.

Table of Contents

Part A — Programming Languages 1. Java 2. Python 3. C++ 4. JavaScript

Part B — Web Technologies 5. HTML 6. CSS 7. React 8. Angular 9. Streamlit

Part C — AI / ML 10. Scikit-learn 11. TensorFlow.js

Part D — Databases 12. MySQL 13. Oracle 14. MongoDB

Part E — Tools 15. Git 16. VS Code 17. Eclipse 18. IntelliJ IDEA

1. Java

1.1 What is it?

Java is a class-based, object-oriented, platform-independent programming language developed by Sun Microsystems (1995), now owned by Oracle. Its core philosophy is “**Write Once, Run Anywhere**” (**WORA**) — Java source code is compiled into an intermediate form called **bytecode** (.class files), which runs on any device that has a **Java Virtual Machine (JVM)**, regardless of the underlying operating system or hardware.

Java is **statically typed**, **strongly typed**, and **automatically memory-managed** (via Garbage Collection). It supports procedural and object-oriented paradigms, and modern versions (8+) support functional-style programming through lambdas and streams.

1.2 When is it used?

- **Enterprise back-end systems** — banking, insurance, and large-scale transactional systems (Java EE / Jakarta EE, Spring Boot).
- **Android app development** — Java (alongside Kotlin) is a primary language for native Android apps.
- **Large-scale distributed systems** — e.g., Hadoop, Kafka, Cassandra are written in Java.
- **Web back-ends / REST APIs** — using Spring Boot, Micronaut, or Jakarta EE.
- **Desktop GUI applications** — using JavaFX or Swing.
- **Embedded and IoT systems** — using Java ME.
- Anywhere you need **portability, strong typing, and a mature ecosystem** with long-term support.

1.3 How is it used? (Typical workflow)

1. Write source code in .java files.
2. Compile with `javac` → produces .class bytecode files.
3. Run using `java ClassName`, which starts the JVM, loads classes via the `ClassLoader`, verifies bytecode, and executes it (interpreted, then JIT-compiled to native machine code for hot code paths).
4. Package classes into a .jar (Java Archive) for distribution, or a .war for web apps.
5. Use build tools like **Maven** or **Gradle** to manage dependencies and automate compile/test/package/deploy.
6. Use a framework (Spring Boot is the most common today) to build REST APIs, wire dependency injection, and connect to databases.

Source (.java) --javac--> Bytecode (.class) --JVM--> Native execution

1.4 Why is it used?

- **Platform independence** via the JVM.
- **Robust and secure**: strong type checking at compile time, exception handling, no explicit pointers, bytecode verification, and a security manager.
- **Automatic memory management** via Garbage Collection — reduces memory leaks compared to manual memory management in C/C++.
- **Rich standard library (JDK)** covering collections, concurrency, I/O, networking, etc.
- **Multithreading built into the language** (Thread, Runnable, java.util.concurrent).
- **Backward compatibility** and huge, mature ecosystem (Spring, Hibernate, Maven).
- **Performance**: JIT compilation makes Java competitive with natively compiled languages for long-running server processes.

1.5 Core Syntax & Examples

Basic structure

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Variables & Data Types

```
int age = 25;                // primitive  
double salary = 55000.50;  
boolean isActive = true;  
char grade = 'A';  
String name = "Alice";      // reference type (object)  
final int MAX = 100;        // constant
```

Control Flow

```
if (age >= 18) {  
    System.out.println("Adult");  
} else {  
    System.out.println("Minor");  
}  
  
for (int i = 0; i < 5; i++) { System.out.println(i); }  
  
int i = 0;  
while (i < 5) { i++; }  
  
switch (grade) {  
    case 'A' -> System.out.println("Excellent");    // Java 14+ arrow syntax  
    default -> System.out.println("Other");  
}
```

OOP — Classes, Inheritance, Interfaces

```
interface Shape {  
    double area();                // abstract method  
}  
  
abstract class AbstractShape implements Shape {  
    protected String name;  
    AbstractShape(String name) { this.name = name; }  
    public abstract double area();  
    public void describe() {  
        System.out.println(name + " area = " + area());  
    }  
}
```

```

class Circle extends AbstractShape {
    private double radius;
    Circle(double radius) {
        super("Circle");
        this.radius = radius;
    }
    @Override
    public double area() { return Math.PI * radius * radius; }
}

public class Main {
    public static void main(String[] args) {
        Shape s = new Circle(5);
        System.out.println(s.area());
    }
}

```

Exception Handling

```

try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Error: " + e.getMessage());
} finally {
    System.out.println("Cleanup runs always");
}

```

Collections & Generics

```

List<String> names = new ArrayList<>();
names.add("Alice");
names.add("Bob");

Map<String, Integer> ages = new HashMap<>();
ages.put("Alice", 30);

for (Map.Entry<String, Integer> entry : ages.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}

```

Streams & Lambdas (Java 8+)

```

List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);
int sumOfSquaresOfEvens = nums.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .reduce(0, Integer::sum);
System.out.println(sumOfSquaresOfEvens);    // 56

```

Multithreading

```
Runnable task = () -> System.out.println("Running in: " + Thread.currentThread().getName());
Thread t = new Thread(task);
t.start();
```

1.6 Basic Interview Questions

Q1. What are the main features of Java? Platform independence, object-oriented, robust, secure, multithreaded, automatic garbage collection, architecture-neutral, high performance via JIT, and dynamic loading.

Q2. What is the difference between JDK, JRE, and JVM? - JVM (Java Virtual Machine): Executes bytecode; provides platform independence. - **JRE (Java Runtime Environment):** JVM + core libraries needed to *run* Java programs. - **JDK (Java Development Kit):** JRE + development tools (compiler javac, debugger) needed to *build* Java programs.

Q3. What is the difference between == and .equals()? == compares references (memory addresses) for objects, and values for primitives. .equals() compares logical/content equality and can be overridden (e.g., String, Integer override it to compare values).

Q4. What is the difference between an abstract class and an interface? An abstract class can have both abstract and concrete methods, constructors, and instance state; a class can extend only one abstract class. An interface (pre-Java 8) could only declare abstract methods; since Java 8 it can have default and static methods, but still no constructors or instance fields (only public static final constants). A class can implement multiple interfaces, enabling multiple inheritance of *type*.

Q5. What are the four OOP pillars, with Java examples? - **Encapsulation:** private fields + public getters/setters. - **Inheritance:** extends keyword. - **Polymorphism:** method overloading (compile-time) and overriding (runtime). - **Abstraction:** abstract classes and interfaces hiding implementation details.

Q6. What is the difference between ArrayList and LinkedList? ArrayList is backed by a dynamic array — O(1) random access, O(n) insert/delete in the middle. LinkedList is a doubly linked list — O(1) insert/delete at known nodes, O(n) random access.

Q7. What is method overloading vs overriding? Overloading: same method name, different parameter list, within the same class (compile-time/static polymorphism). Overriding: subclass redefines a superclass method with the same signature (runtime/dynamic polymorphism), enabled via inheritance and the @Override annotation.

Q8. What is the purpose of the static keyword? static members belong to the class rather than any instance — shared across all objects. Static methods can't access instance (non-static) members directly and cannot be overridden (they can be hidden).

1.7 Advanced Interview Questions

Q1. Explain the Java Memory Model — Heap vs Stack. Each thread has its own **Stack**, storing method call frames, local variables, and object references (LIFO). The **Heap** is shared across all threads and stores actual objects and arrays. The heap is further divided into Young Generation (Eden + Survivor spaces) and Old Generation for generational garbage collection; there's also Metaspace (post Java 8) for class metadata.

Q2. How does Garbage Collection work in Java? Name some GC algorithms. GC automatically reclaims heap memory occupied by objects with no reachable references. Most

JVMs use **generational GC**: new objects are allocated in Eden; survivors are promoted to Survivor spaces, then Old Gen after surviving enough minor GCs. Algorithms include **Serial GC**, **Parallel GC**, **CMS (deprecated)**, **G1GC (default since Java 9)**, and **ZGC/Shenandoah** (low-latency, near-pauseless collectors for large heaps).

Q3. What is the difference between String, StringBuilder, and StringBuffer? String is immutable — every modification creates a new object (also uses the String Pool for literal caching/interning). StringBuilder is mutable and **not thread-safe**, offering better performance for single-threaded string manipulation. StringBuffer is mutable and thread-safe (synchronized methods) but slower due to synchronization overhead.

Q4. Explain HashMap internal working. A HashMap stores entries in an array of buckets. hashCode() of the key determines the bucket index (hash & (n-1)). Collisions within a bucket are handled by a linked list; since Java 8, if a bucket's list exceeds a threshold (8) and the table is large enough, it's converted into a **red-black tree** for O(log n) worst case instead of O(n). Resizing (rehashing) happens when load factor (default 0.75) is exceeded, doubling capacity.

Q5. What is the difference between checked and unchecked exceptions? Checked exceptions (subclasses of Exception, excluding RuntimeException) must be either caught or declared with throws — enforced at compile time (e.g., IOException). Unchecked exceptions (subclasses of RuntimeException, e.g., NullPointerException) are not enforced at compile time and typically indicate programming bugs.

Q6. Explain the volatile keyword and how it differs from synchronized. volatile ensures visibility of a variable's latest value across threads by preventing CPU/JIT caching/reordering (forces reads/writes to main memory) but does **not** provide atomicity for compound operations (e.g., i++). synchronized provides both **mutual exclusion** (only one thread in the block at a time) and **visibility**, but at the cost of higher overhead and possible contention/deadlocks.

Q7. What are functional interfaces and how do lambdas relate to them? A functional interface has exactly one abstract method (e.g., Runnable, Callable, Comparator, or custom ones annotated @FunctionalInterface). A lambda expression is essentially an anonymous implementation of a functional interface's single abstract method, compiled using invokedynamic rather than generating a full anonymous class file.

Q8. What is the difference between final, finally, and finalize()? final is a modifier making a variable constant, a method non-overridable, or a class non-inheritable. finally is a block that always executes after try/catch (used for cleanup like closing resources). finalize() is a deprecated Object method the GC used to call before reclaiming an object — replaced by try-with-resources and Cleaner/PhantomReference in modern Java.

Q9. Explain Java's try-with-resources. Introduced in Java 7, it automatically closes any resource implementing AutoCloseable/Closeable at the end of the block, even if an exception occurs, eliminating verbose manual finally cleanup:

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {  
    System.out.println(br.readLine());  
}
```

Q10. What are Java Records (Java 16+)? Records are immutable data carrier classes that auto-generate a canonical constructor, equals(), hashCode(), toString(), and accessor methods from a compact declaration:

```
record Point(int x, int y) {}
```

2. Python

2.1 What is it?

Python is a high-level, interpreted, dynamically-typed, general-purpose programming language created by Guido van Rossum (1991), known for its clean, readable syntax (significant whitespace/indentation instead of braces). It supports multiple paradigms — procedural, object-oriented, and functional — and ships with a massive standard library (“batteries included”) plus an enormous third-party ecosystem (PyPI).

2.2 When is it used?

- **Data Science & Machine Learning** — pandas, NumPy, scikit-learn, TensorFlow, PyTorch.
- **Web development** — Django, Flask, FastAPI.
- **Automation & scripting** — system administration, task automation, web scraping.
- **Rapid prototyping** — because of concise syntax and quick iteration.
- **AI/LLM tooling** — most modern AI frameworks (LangChain, HuggingFace) are Python-first.
- **Scientific computing** — SciPy, Jupyter notebooks.
- **Testing/QA automation** — Selenium, PyTest.

2.3 How is it used?

1. Write .py scripts/modules.
2. Run directly via the CPython interpreter (`python script.py`) — no separate compile step needed by the developer (it’s compiled to bytecode .pyc internally and interpreted by the Python VM).
3. Manage dependencies with pip and virtual environments (venv, conda) to isolate project dependencies.
4. Structure larger projects into packages/modules using import.
5. Use frameworks (Flask/Django for web, pandas/NumPy for data) to accelerate development.

2.4 Why is it used?

- **Readability and low learning curve** — ideal for beginners and fast development.
- **Huge ecosystem**, especially dominant in **AI/ML/data science**.
- **Dynamic typing** enables fast prototyping (though it trades off some compile-time safety).
- **Cross-platform** and glue-language capability — easily interfaces with C/C++/Java libraries.
- **Strong community support** and extensive documentation.
- **Interpreted nature** allows interactive development (REPL, Jupyter).

2.5 Core Syntax & Examples

Basics

```
name = "Alice"           # dynamically typed
age = 30
is_active = True
height = 5.6
```

```
print(f"{name} is {age} years old")    # f-string formatting
```

Control Flow

```
if age >= 18:
    print("Adult")
elif age >= 13:
    print("Teen")
else:
    print("Child")

for i in range(5):
    print(i)

n = 0
while n < 5:
    n += 1
```

Data Structures

```
my_list = [1, 2, 3]                # mutable, ordered
my_tuple = (1, 2, 3)                # immutable, ordered
my_set = {1, 2, 3}                  # unordered, unique
my_dict = {"a": 1, "b": 2}          # key-value pairs

squares = [x**2 for x in range(10) if x % 2 == 0]    # list comprehension
```

Functions

```
def greet(name, greeting="Hello"):
    return f"{greeting}, {name}!"

print(greet("Bob"))                  # default arg
print(greet("Bob", greeting="Hi"))

add = lambda a, b: a + b             # lambda
print(add(3, 4))

def variadic(*args, **kwargs):
    print(args, kwargs)
```

OOP

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        raise NotImplementedError
```

```
class Dog(Animal):
    def speak(self):
        return f"{self.name} says Woof!"

d = Dog("Rex")
print(d.speak())
```

Exception Handling

```
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error: {e}")
finally:
    print("Cleanup")
```

Decorators, Generators, Context Managers

```
def timer(func):
    def wrapper(*args, **kwargs):
        import time
        start = time.time()
        result = func(*args, **kwargs)
        print(f"{func.__name__} took {time.time() - start:.4f}s")
        return result
    return wrapper

@timer
def slow_square(n):
    return n * n

def count_up_to(n):
    i = 1
    while i <= n:
        yield i          # generator - lazy evaluation
        i += 1

with open("file.txt", "w") as f:  # context manager
    f.write("Hello")
```

2.6 Basic Interview Questions

Q1. What are Python's key features? Interpreted, dynamically typed, high-level, supports multiple paradigms, automatic memory management (reference counting + garbage collection), extensive standard library, and a huge third-party package ecosystem via pip.

Q2. What is the difference between a list, tuple, and set? List: ordered, mutable, allows duplicates. Tuple: ordered, immutable, allows duplicates (hashable if elements are hashable, so usable as dict keys). Set: unordered, mutable, no duplicates, backed by a hash table for O(1) average membership tests.

Q3. What is the difference between is and ==? == compares value equality (calls `__eq__`); is compares identity (whether both references point to the exact same object in memory).

Q4. What are `*args` and `**kwargs`? `*args` collects extra positional arguments into a tuple; `**kwargs` collects extra keyword arguments into a dictionary, enabling variadic function signatures.

Q5. What is PEP 8? The official Python style guide covering naming conventions, indentation (4 spaces), line length, and formatting best practices for readable, consistent code.

Q6. What is the difference between deep copy and shallow copy? A shallow copy (`copy.copy()`) creates a new object but inserts references to the same nested objects. A deep copy (`copy.deepcopy()`) recursively copies all nested objects, so changes to the copy don't affect the original.

Q7. What are Python decorators? Functions that wrap another function/class to extend its behavior without modifying its source code, using the `@decorator_name` syntax — commonly used for logging, timing, authentication, and caching (`@functools.lru_cache`).

Q8. What is list comprehension and why use it? A concise syntax to build lists from iterables in a single readable line (`[expr for item in iterable if condition]`), generally faster and more Pythonic than an equivalent explicit for loop with `.append()`.

2.7 Advanced Interview Questions

Q1. What is the Global Interpreter Lock (GIL) and its implications? The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time, even on multi-core machines. This means threading doesn't achieve true parallelism for CPU-bound tasks (though it works well for I/O-bound tasks, since the GIL is released during I/O waits). For CPU-bound parallelism, developers use multiprocessing (separate processes, separate memory/interpreter, no shared GIL) or C-extension libraries (NumPy releases the GIL internally for heavy computation).

Q2. Explain Python's memory management model. CPython primarily uses **reference counting**: every object has a count of references pointing to it; when it drops to zero, the memory is freed immediately. To handle **reference cycles** (objects referencing each other) that reference counting alone can't collect, Python has a supplementary **generational garbage collector** (`gc` module) that periodically detects and cleans up unreachable cycles.

Q3. What are Python's Method Resolution Order (MRO) rules for multiple inheritance? Python uses the **C3 linearization algorithm** to compute a consistent method resolution order in multiple inheritance, ensuring a class always appears before its parents and preserving the order parents are listed. You can inspect it with `ClassName.__mro__` or `ClassName.mro()`.

Q4. What is the difference between `@staticmethod`, `@classmethod`, and instance methods? Instance methods take `self` and operate on instance state. `@classmethod` takes `cls` and operates on the class itself (useful for alternative constructors/factory methods). `@staticmethod` takes neither — it's a plain function namespaced inside the class for organizational purposes, with no access to instance or class state.

Q5. Explain generators vs iterators, and why generators are memory-efficient. An iterator is any object implementing `__iter__` and `__next__`. A generator is a special iterator created via a function using `yield`, or a generator expression (`x for x in ...`). Generators produce values **lazily, one at a time**, rather than materializing the whole sequence in memory — critical when processing very large or infinite data streams.

Q6. What are Python descriptors and where are they used? A descriptor is an object implementing `__get__`, `__set__`, or `__delete__`, controlling attribute access on other classes. They power Python's own property, methods (functions are non-data descriptors that bind `self`), and ORMs like Django's model fields.

Q7. Explain `__slots__` and its benefit. By default, Python instances store attributes in a per-instance `__dict__`, which has memory overhead. Declaring `__slots__ = ('x', 'y')` in a class tells Python to allocate fixed slots for those attributes instead of a dict, reducing memory footprint significantly for classes with millions of instances, at the cost of losing dynamic attribute assignment (unless `__dict__` is explicitly kept).

Q8. What is monkey patching? Dynamically modifying or extending a module/class at runtime (e.g., reassigning a method on an existing class) without touching its source code. Useful for testing (mocking) but risky in production due to reduced predictability.

Q9. How does Python's asyncio model concurrency differently from threading? `asyncio` uses a **single-threaded event loop** with cooperative multitasking: coroutines (`async def`) voluntarily yield control at `await` points, allowing many I/O-bound tasks to be interleaved efficiently without OS thread context-switching overhead or GIL contention — but a single blocking (CPU-bound) call can stall the entire event loop.

Q10. Explain how Python's `sorted()` / `list.sort()` algorithm works. Python uses **Timsort**, a hybrid stable sorting algorithm derived from merge sort and insertion sort, with $O(n \log n)$ worst case and $O(n)$ best case (for nearly sorted data) — it exploits existing order (“runs”) in real-world data for efficiency.

3. C++

3.1 What is it?

C++ is a general-purpose, compiled, statically-typed language created by Bjarne Stroustrup as an extension of C, adding object-oriented, generic, and (later) functional programming features while retaining low-level memory control. It compiles directly to native machine code (no VM), giving it near-hardware performance.

3.2 When is it used?

- **System/embedded programming** — operating systems, device drivers, firmware.
- **Game engines** — Unreal Engine, most AAA game engines are C++.
- **High-frequency trading / low-latency systems** — where microseconds matter.
- **Competitive programming** — because of speed and control over data structures (STL).
- **Performance-critical libraries** — e.g., parts of TensorFlow, OpenCV, database engines are written in C++.
- **Real-time systems and robotics.**

3.3 How is it used?

1. Write .cpp/.h files.
2. Compile with a compiler (g++, clang++, MSVC) → object files (.o), then **link** against libraries to produce an executable.
3. Manual or RAII-based memory management: new/delete, or (preferably) smart pointers (unique_ptr, shared_ptr) that free memory automatically when they go out of scope.
4. Use the **STL (Standard Template Library)** for data structures (vector, map, set) and algorithms (sort, find).
5. Use build systems like **CMake** or **Make** to manage multi-file, multi-platform builds.

Source (.cpp) --compiler--> Object files (.o) --linker--> Native executable

3.4 Why is it used?

- **Performance:** compiled directly to native code, no runtime interpreter/VM overhead.
- **Fine-grained control** over memory and hardware resources.
- **Zero-cost abstractions:** high-level features (templates, OOP) compile down with minimal/no runtime cost.
- **Deterministic resource management** via RAII (Resource Acquisition Is Initialization) — no unpredictable GC pauses.
- **Rich standard library (STL)** with efficient, well-tested containers/algorithms.
- **Backward compatible with C**, enabling integration with existing low-level codebases.

3.5 Core Syntax & Examples

Basics

```
#include <iostream>
using namespace std;

int main() {
    int age = 25;
    double salary = 50000.5;
```

```

string name = "Alice";
cout << name << " is " << age << " years old" << endl;
return 0;
}

```

Control Flow

```

if (age >= 18) { cout << "Adult"; } else { cout << "Minor"; }

for (int i = 0; i < 5; i++) cout << i << " ";

int i = 0;
while (i < 5) { i++; }

```

Pointers & References

```

int x = 10;
int* ptr = &x;      // pointer holds address of x
int& ref = x;        // reference is an alias for x

cout << *ptr << endl; // dereference -> 10
*ptr = 20;            // modifies x through pointer

```

Classes & OOP

```

class Shape {
protected:
    string name;
public:
    Shape(string n) : name(n) {}
    virtual double area() const = 0; // pure virtual -> abstract class
    virtual ~Shape() {}              // virtual destructor
};

class Circle : public Shape {
    double radius;
public:
    Circle(double r) : Shape("Circle"), radius(r) {}
    double area() const override { return 3.14159 * radius * radius; }
};

int main() {
    Shape* s = new Circle(5);
    cout << s->area() << endl;
    delete s;
    return 0;
}

```


Smart Pointers (Modern C++)

```
#include <memory>

std::unique_ptr<Circle> c1 = std::make_unique<Circle>(5); // exclusive ownership
std::shared_ptr<Circle> c2 = std::make_shared<Circle>(3); // reference-counted ownership
```

Templates (Generic Programming)

```
template <typename T>
T maxVal(T a, T b) {
    return (a > b) ? a : b;
}

cout << maxVal<int>(3, 7);
cout << maxVal<double>(3.5, 2.1);
```

STL

```
#include <vector>
#include <algorithm>

vector<int> nums = {5, 3, 8, 1};
sort(nums.begin(), nums.end());
for (int n : nums) cout << n << " ";
```

3.6 Basic Interview Questions

Q1. What is the difference between C and C++? C is procedural; C++ adds object-oriented (classes, inheritance, polymorphism), generic programming (templates), exception handling, and a richer standard library (STL), while remaining largely backward-compatible with C.

Q2. What is the difference between a pointer and a reference? A pointer is a variable holding a memory address; it can be reassigned, can be null, and needs explicit dereferencing (*). A reference is an alias for an existing variable; it must be initialized at declaration, cannot be null or resealed, and is used implicitly (no * needed).

Q3. What is a constructor and destructor? A constructor is a special method automatically called when an object is created, used for initialization. A destructor (~ClassName()) is automatically called when an object goes out of scope or is deleted, used for cleanup (freeing resources).

Q4. What is function overloading? Defining multiple functions with the same name but different parameter lists (type/number), resolved at compile time based on the arguments passed.

Q5. What is the difference between struct and class in C++? Functionally nearly identical — the only difference is default access: struct members are public by default; class members are private by default.

Q6. What is operator overloading? Redefining the behavior of operators (+, ==, <<, etc.) for user-defined types, allowing objects to be used with natural syntax, e.g. c1 + c2 for two Complex objects.

Q7. What is the difference between stack and heap memory? Stack: automatic, fast, fixed-size, LIFO allocation for local variables — freed automatically when scope ends. Heap: dynamic, manually managed (`new/delete`) or smart-pointer-managed, larger but slower to allocate, persists until explicitly freed.

3.7 Advanced Interview Questions

Q1. Explain virtual functions and vtables. A virtual function enables **runtime polymorphism**: calling an overridden method through a base class pointer/reference invokes the derived class’s implementation. The compiler implements this via a **vtable** (virtual table) — a per-class array of function pointers — and each object of a polymorphic class carries a hidden **vptr** pointing to its class’s vtable, so the call is resolved indirectly through this table at runtime.

Q2. What is RAII and why does it matter? RAII (Resource Acquisition Is Initialization) ties resource lifetime (memory, file handles, locks, sockets) to object lifetime: acquire the resource in the constructor, release it in the destructor. Because C++ guarantees destructors run when objects go out of scope (including during stack unwinding from exceptions), RAII gives deterministic, exception-safe resource cleanup — the foundation for smart pointers, `lock_guard`, and file streams.

Q3. Difference between `unique_ptr`, `shared_ptr`, and `weak_ptr`. `unique_ptr`: sole, non-copyable ownership; near-zero overhead; moved (not copied) to transfer ownership. `shared_ptr`: shared ownership via atomic reference counting — object destroyed when the last `shared_ptr` is destroyed; has overhead from the control block and atomic increments/decrements. `weak_ptr`: a non-owning observer of a `shared_ptr`-managed object, used to break reference cycles (e.g., parent/child mutual references) without keeping the object alive.

Q4. What is move semantics and rvalue references (C++11)? Move semantics let resources (heap buffers, etc.) be **transferred** from a temporary/expiring object to a new one instead of deep-copied, via **rvalue references** (T&E) and move constructors/assignment operators (`std::move`). This dramatically improves performance for containers of heavy objects (e.g., returning large vectors by value).

Q5. What is the diamond problem and how does C++ solve it? It arises with multiple inheritance when two base classes both inherit from a common ancestor, and a derived class inherits from both — creating ambiguous duplicate copies of the ancestor’s members. C++ solves it with **virtual inheritance** (`class B : virtual public A`), ensuring only a single shared instance of the common base exists in the derived object.

Q6. Explain templates vs generics (conceptually) and template metaprogramming. Unlike Java/C# generics (type-erased or reified at runtime), C++ templates are resolved at **compile time** — the compiler generates a separate specialized version of the function/class for each distinct type used (monomorphization), giving zero runtime overhead but increasing binary size (“code bloat”). Template metaprogramming exploits this to perform computation *at compile time* (e.g., `constexpr` functions, SFINAE, and C++20 Concepts for constraining template parameters).

Q7. What is undefined behavior (UB), and give examples. Code whose behavior the C++ standard does not define — the compiler is free to do anything (crash, produce wrong results, or even “optimize away” seemingly correct code). Examples: dereferencing a null/dangling pointer, signed integer overflow, buffer overrun, using an uninitialized variable, and data races on non-atomic variables across threads.

Q8. What is the Rule of Three/Five/Zero? If a class manages a resource manually, the **Rule of Three** says it should define a destructor, copy constructor, and copy assignment

operator together (if you need one, you likely need all three). C++11's **Rule of Five** adds the move constructor and move assignment operator. The **Rule of Zero** — the modern best practice — says to avoid managing raw resources directly at all; instead compose the class from RAII types (like `unique_ptr`, `vector`, `string`) so the compiler-generated special member functions are correct automatically.

4. JavaScript

4.1 What is it?

JavaScript (JS) is a high-level, interpreted (JIT-compiled), dynamically-typed, prototype-based, multi-paradigm scripting language, originally created for adding interactivity to web pages inside browsers. It is now used both **client-side** (in browsers, via the DOM) and **server-side** (via Node.js), and is standardized as **ECMAScript (ES)**.

4.2 When is it used?

- **Front-end web interactivity** — DOM manipulation, form validation, animations, SPAs.
- **Back-end development** — Node.js, Express.js for building APIs/servers.
- **Mobile app development** — React Native.
- **Desktop apps** — Electron (VS Code itself is built on Electron).
- **Full-stack JavaScript** — using the same language across the entire stack (MERN/MEAN).

4.3 How is it used?

1. Embed `<script>` in HTML or link an external `.js` file for the browser to parse and execute.
2. The browser's JS engine (e.g., V8 in Chrome/Node.js) parses, compiles (JIT), and executes the code on a **single-threaded event loop**.
3. Asynchronous operations (network calls, timers) are handled via callbacks, Promises, or `async/await`, offloaded to Web APIs / libuv (Node) and queued back onto the event loop's callback/microtask queue.
4. On the server, Node.js runs JS outside the browser using the same V8 engine plus additional APIs (file system, networking).
5. Use package managers (npm/yarn/pnpm) and bundlers (Webpack/Vite) for larger projects.

4.4 Why is it used?

- **Ubiquity**: it is the only language natively executed by every web browser.
- **Non-blocking, event-driven I/O** model — excellent for high-concurrency I/O-bound applications (Node.js).
- **Huge ecosystem** (npm — largest package registry in the world).
- **Full-stack capability** with one language on client and server.
- **First-class functions and closures** enable powerful functional and asynchronous patterns.
- **Fast iteration** — no compile step needed for development (though modern tooling often transpiles/bundles).

4.5 Core Syntax & Examples

Variables & Types

```
let age = 25;           // block-scoped, mutable
const name = "Alice";  // block-scoped, constant reference
var legacy = "old";     // function-scoped (avoid in modern code)

console.log(typeof age); // "number"
```

Functions & Arrow Functions

```
function greet(name) {  
  return `Hello, ${name}!`;  
}  
  
const greetArrow = (name) => `Hello, ${name}!`; // arrow function, lexical `this`  
  
const add = (a, b = 10) => a + b; // default parameter
```

Objects & Arrays

```
const person = { name: "Alice", age: 30 };  
const { name, age } = person; // destructuring  
  
const nums = [1, 2, 3, 4, 5];  
const doubled = nums.map(n => n * 2);  
const evens = nums.filter(n => n % 2 === 0);  
const sum = nums.reduce((acc, n) => acc + n, 0);
```

Asynchronous JS

```
// Promise  
fetch("https://api.example.com/data")  
  .then(response => response.json())  
  .then(data => console.log(data))  
  .catch(error => console.error(error));  
  
// async/await  
async function getData() {  
  try {  
    const response = await fetch("https://api.example.com/data");  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.error(error);  
  }  
}
```

Classes & Prototypes

```
class Animal {  
  constructor(name) {  
    this.name = name;  
  }  
  speak() {  
    return `${this.name} makes a sound.`;  
  }  
}  
  
class Dog extends Animal {
```

```

    speak() {
        return `${this.name} barks.`;
    }
}

const d = new Dog("Rex");
console.log(d.speak());

```

Closures

```

function counter() {
    let count = 0;
    return function () {
        count++;
        return count;
    };
}

const increment = counter();
console.log(increment()); // 1
console.log(increment()); // 2 (count is preserved via closure)

```

4.6 Basic Interview Questions

Q1. What is the difference between var, let, and const? var is function-scoped, hoisted and initialized as undefined, and can be redeclared. let is block-scoped, hoisted but not initialized (temporal dead zone), and can be reassigned but not redeclared in the same scope. const is block-scoped like let but the binding cannot be reassigned (though object/array contents referenced by a const can still be mutated).

Q2. What is the difference between == and ===? == performs type coercion before comparing values (e.g., "5" == 5 is true). === (strict equality) compares both value and type without coercion ("5" === 5 is false). Best practice is to always use ===.

Q3. What is hoisting? JavaScript's behavior of moving variable and function *declarations* to the top of their scope during compilation. function declarations are fully hoisted (usable before their line); var declarations are hoisted but initialized as undefined; let/const are hoisted into a "temporal dead zone" and throw a ReferenceError if accessed before declaration.

Q4. What is a closure? A function that "remembers" and can access variables from its enclosing (outer) lexical scope even after that outer function has finished executing — used for data privacy, factory functions, and callbacks.

Q5. What is the event loop? The mechanism that allows JavaScript's single-threaded runtime to handle asynchronous operations: the **call stack** executes synchronous code; long-running/async operations are delegated to Web APIs/Node APIs; once complete, their callbacks are placed in the **task queue** (macrotasks) or **microtask queue** (Promises); the event loop continuously checks if the call stack is empty and, if so, pushes the next queued callback (microtasks first) onto the stack.

Q6. What is the difference between null and undefined? undefined means a variable has been declared but not assigned a value (JS's default). null is an intentional assignment representing "no value" / empty, explicitly set by the developer.

Q7. What are template literals? String literals delimited by backticks (`) that support embedded expressions via \${expr} and multi-line strings without explicit \n.

4.7 Advanced Interview Questions

Q1. Explain the JavaScript prototype chain. Every JS object has an internal `[[Prototype]]` link (accessible via `Object.getPrototypeOf()` or the deprecated `__proto__`) to another object. When accessing a property, the engine looks it up on the object itself first, then walks up the prototype chain until it's found or the chain ends at `null`. `class` syntax is syntactic sugar over this prototype-based inheritance model.

Q2. Explain this binding rules in JavaScript. this is determined by **how a function is called**, not where it's defined (except for arrow functions): (1) plain function call $\rightarrow$ this is undefined (strict mode) or global object; (2) method call (`obj.method()`) $\rightarrow$ this is `obj`; (3) constructor call (`new Fn()`) $\rightarrow$ this is the newly created object; (4) explicit binding via `call`/`apply`/`bind`; (5) arrow functions don't have their own this — they lexically inherit it from their enclosing scope at definition time.

Q3. Microtasks vs Macrotasks — which runs first? Promises (`.then`, `async/await` continuations), and `queueMicrotask` are **microtasks**; `setTimeout`, `setInterval`, and I/O callbacks are **macrotasks**. After each macrotask completes, the event loop drains the **entire microtask queue** before rendering or picking up the next macrotask — so a `Promise.resolve().then()` always fires before a `setTimeout(fn, 0)`.

Q4. Explain debouncing vs throttling. Both limit how often a function runs in response to frequent events (scroll, resize, keypress). **Debouncing** delays execution until a pause of silence (e.g., wait 300ms after the user stops typing before firing a search). **Throttling** ensures the function runs at most once per fixed interval regardless of how many events fire (e.g., at most once every 300ms during continuous scrolling).

Q5. What is the difference between deep and shallow copy in JS, and common pitfalls? `Spread` (`{...obj}`) and `Object.assign()` create **shallow copies** — nested objects/arrays are still shared by reference. A true deep copy requires recursive cloning, `structuredClone()` (modern browsers/Node), or libraries like `lodash's cloneDeep`. A common bug is mutating a nested object in a “copied” state object in `React/Redux`, believing it's isolated.

Q6. Explain currying and function composition. Currying transforms a function of multiple arguments into a sequence of functions each taking a single argument: `const add = a => b => a + b`; `. Composition chains functions so the output of one feeds into the next, enabling declarative, reusable data pipelines — foundational to functional-programming-style JS.`

Q7. What are WeakMap and WeakSet, and why use them? Like `Map/Set`, but keys (`WeakMap`) or values (`WeakSet`) must be objects and are held with a **weak reference** — meaning they don't prevent garbage collection if there are no other references to them. Useful for associating metadata with objects (e.g., DOM nodes) without causing memory leaks.

Q8. Explain the differences between call, apply, and bind. All three explicitly set this for a function. `call(thisArg, arg1, arg2, ...)` invokes the function immediately with individual arguments. `apply(thisArg, [argsArray])` invokes immediately with arguments as an array. `bind(thisArg, ...)` returns a **new function** with this permanently bound, without invoking it immediately — commonly used for event handlers and partial application.

Q9. What is the difference between synchronous and asynchronous module systems (CommonJS vs ES Modules)? `CommonJS` (`require/module.exports`, Node's original system) loads modules **synchronously** at runtime. `ES Modules` (`import/export`) are statically analyzable, support **tree-shaking** (dead code elimination by bundlers), asynchronous loading, and are now natively supported in both browsers and modern Node.js.

Q10. What causes memory leaks in JavaScript and how do you avoid them? Common causes: forgotten event listeners/timers holding references to detached DOM nodes, global variable accumulation, closures unintentionally retaining large objects, and uncleared caches. Mitigations: remove listeners/timers on cleanup (e.g., in `React's useEffect` cleanup function),

use `WeakMap/WeakSet` for auxiliary data, and profile with browser DevTools' memory/heap snapshot tools.

5. HTML

5.1 What is it?

HTML (HyperText Markup Language) is the standard **markup language** (not a programming language) used to structure content on the web. It uses a system of **elements/tags** that define the meaning and structure of content — headings, paragraphs, links, images, forms, tables — which the browser parses into the **DOM (Document Object Model)**, a tree structure that CSS styles and JavaScript manipulates.

5.2 When is it used?

- Every web page uses HTML as its structural foundation — it is non-negotiable for any website or web app.
- **Semantic structuring** of content for accessibility and SEO (<header>, <nav>, <main>, <article>, <footer>).
- **Forms** for user input collection.
- **Email templates** (a restricted subset of HTML/CSS).
- As the **output target** for every front-end framework (React, Angular, etc. ultimately render to HTML/DOM).

5.3 How is it used?

1. Write a .html file with a root <html> element containing <head> (metadata, title, links to CSS/JS) and <body> (visible content).
2. The browser's HTML parser reads the file top-to-bottom, builds the **DOM tree**.
3. The browser separately parses CSS into the **CSSOM**, combines DOM + CSSOM into a **render tree**, then performs **layout (reflow)** and **paint** to display pixels on screen.
4. JavaScript can query/manipulate the DOM at runtime (document.querySelector, etc.) to make pages dynamic.

5.4 Why is it used?

- It is the **universal, browser-native structural language** — every browser understands it without additional tooling.
- Provides **semantic meaning** to content, which improves accessibility (screen readers) and SEO.
- Forms the **foundation** all styling (CSS) and behavior (JS) attach to.
- Simple, declarative, human-readable syntax.

5.5 Core Syntax & Examples

Basic Document Structure

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Page</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

```

<header>
  <h1>Welcome</h1>
  <nav>
    <a href="#home">Home</a>
    <a href="#about">About</a>
  </nav>
</header>

<main>
  <article>
    <h2>Article Title</h2>
    <p>This is a paragraph with <strong>bold</strong> and <em>italic</em> text.</p>
    
    <a href="https://example.com">Visit Example</a>
  </article>
</main>

<footer>
  <p>&copy; 2026 My Website</p>
</footer>
<script src="script.js"></script>
</body>
</html>

```

Forms

```

<form action="/submit" method="POST">
  <label for="email">Email:</label>
  <input type="email" id="email" name="email" required>

  <label for="age">Age:</label>
  <input type="number" id="age" name="age" min="0">

  <select name="country">
    <option value="in">India</option>
    <option value="us">USA</option>
  </select>

  <button type="submit">Submit</button>
</form>

```

Tables & Lists

```

<table>
  <thead>
    <tr><th>Name</th><th>Age</th></tr>
  </thead>
  <tbody>
    <tr><td>Alice</td><td>30</td></tr>
  </tbody>
</table>

```

```
<ul>
  <li>Item 1</li>
  <li>Item 2</li>
</ul>
```

5.6 Basic Interview Questions

Q1. What is the difference between HTML elements and tags? A tag is the markup syntax itself (<p>, </p>); an element is the tag plus its content and attributes together (<p>Hello</p> is the element).

Q2. What is semantic HTML and why does it matter? Using tags that convey meaning about their content (<article>, <nav>, <section>) instead of generic <div>s for everything. It improves accessibility for screen readers, SEO ranking, and code readability/maintainability.

Q3. What is the difference between <div> and ? <div> is a block-level element (takes full width, starts on a new line) used for grouping larger sections. is an inline element used for styling/grouping small pieces of text within a line, without breaking flow.

Q4. What is the difference between id and class attributes? id must be unique per page and is used for a single specific element (also usable as a CSS/JS hook and URL fragment target). class can be reused across multiple elements to apply shared styling/behavior.

Q5. What are semantic form input types? HTML5 introduced types like email, number, date, tel, url, range — enabling built-in browser validation and appropriate mobile keyboards, instead of relying purely on generic text inputs plus JS validation.

Q6. What is the purpose of the alt attribute on ? Provides alternative text describing the image for screen readers (accessibility) and displays if the image fails to load; also used by search engines for image indexing.

5.7 Advanced Interview Questions

Q1. Explain the critical rendering path. The sequence a browser follows to convert HTML/CSS/JS into pixels: (1) parse HTML → **DOM**; (2) parse CSS → **CSSOM**; (3) combine DOM + CSSOM → **Render Tree** (visible elements only); (4) **Layout/Reflow** — compute exact position/size of every element; (5) **Paint** — fill in pixels; (6) **Composite** — layers are combined by the GPU. Render-blocking resources (synchronous <script> in <head>, external CSS) can delay this pipeline, which is why scripts are often deferred/async and CSS is loaded early.

Q2. What is the difference between defer and async on <script> tags? Both allow the script to download without blocking HTML parsing. async executes the script **as soon as it finishes downloading**, potentially interrupting parsing, and execution order relative to other async scripts is not guaranteed. defer executes the script **only after HTML parsing is complete**, and multiple defer scripts run in the order they appear in the document — generally the safer default for scripts that touch the DOM.

Q3. How does the browser handle DOM reflow and repaint, and why are they expensive? **Reflow (layout)** recalculates element geometry (size/position) when the DOM structure or certain CSS properties change (e.g., width, adding/removing elements) — it can cascade to affect the whole document tree, so it's expensive. **Repaint** just redraws pixels (e.g., color change) without geometry changes — cheaper than reflow. Reading layout properties (like offsetHeight) right after writing styles forces **synchronous layout thrashing**; batching DOM reads/writes avoids this performance pitfall.

Q4. What are Web Components, and what standards make them up? A browser-native way to create reusable, encapsulated custom elements without a framework, built on

three standards: **Custom Elements** (define new HTML tags with JS classes), **Shadow DOM** (encapsulated, style-isolated DOM subtree), and **HTML Templates** (<template>/<slot> for reusable inert markup).

Q5. Explain accessibility (a11y) considerations beyond semantic tags — ARIA. When native semantic HTML can't fully express a UI pattern (custom widgets like tabs, modals, dropdowns), **ARIA (Accessible Rich Internet Applications)** attributes (role, aria-label, aria-expanded, aria-live) communicate state/roles to assistive technologies. The general rule ("No ARIA is better than bad ARIA") is to prefer native semantic elements first, and only add ARIA when native HTML can't express the needed pattern.

6. CSS

6.1 What is it?

CSS (Cascading Style Sheets) is a **style sheet language** used to describe the visual presentation of HTML documents — colors, layout, fonts, spacing, animations, and responsiveness — separating content (HTML) from presentation (CSS).

6.2 When is it used?

- Styling every web page: colors, typography, spacing, layout.
- **Responsive design** — adapting layouts to different screen sizes (mobile, tablet, desktop) via media queries.
- **Animations & transitions** for interactive UI feedback.
- **Component-level styling** in modern frameworks (CSS Modules, styled-components, Tailwind utility classes).
- **Print stylesheets**, dark mode theming (via CSS variables/prefers-color-scheme).

6.3 How is it used?

1. Define rules as selector { property: value; }.
2. Attach CSS to HTML via: inline (style="..." attribute — highest specificity, avoid for maintainability), internal (<style> in <head>), or external (<link rel="stylesheet" href="style.css"> — best practice, cacheable, separates concerns).
3. The browser parses CSS into the **CSSOM**, matches selectors against DOM elements, and resolves the final computed style for each element using the **cascade** (source order, specificity, importance) and **inheritance**.
4. Combined with the DOM to build the render tree for layout and painting.

6.4 Why is it used?

- **Separation of concerns:** keeps structure (HTML) independent from presentation (CSS), improving maintainability.
- **Reusability:** one stylesheet can style many pages consistently.
- **Responsive/adaptive design** across devices via media queries and flexible layout models (Flexbox, Grid).
- **Performance:** cached external stylesheets reduce redundant downloads; GPU-accelerated properties (transform, opacity) enable smooth animations.

6.5 Core Syntax & Examples

Selectors & Basic Rules

```
/* Type selector */
p { color: #333; font-size: 16px; }

/* Class & ID selectors */
.card { padding: 16px; border-radius: 8px; }
#header { background: navy; }

/* Combinators */
.card > .title { font-weight: bold; }      /* direct child */
.card p { margin: 0; }                    /* descendant */
```

```

.btn:hover { background: darkblue; }      /* pseudo-class */
.item::before { content: "→ "; }         /* pseudo-element */

/* Attribute selector */
input[type="email"] { border-color: blue; }

```

Box Model

```

.box {
  width: 200px;
  padding: 16px;      /* space inside border */
  border: 2px solid #ccc;
  margin: 12px;       /* space outside border */
  box-sizing: border-box; /* width includes padding+border */
}

```

Flexbox

```

.container {
  display: flex;
  justify-content: space-between; /* main-axis alignment */
  align-items: center;           /* cross-axis alignment */
  gap: 16px;
}
.item { flex: 1; } /* grow to fill available space equally */

```

Grid

```

.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: auto;
  gap: 20px;
}
.grid-item { grid-column: span 2; }

```

Responsive Design (Media Queries)

```

.container { width: 100%; }

@media (min-width: 768px) {
  .container { width: 750px; margin: 0 auto; }
}

@media (min-width: 1200px) {
  .container { width: 1140px; }
}

```

CSS Variables & Animation

```
:root {
  --primary-color: #3b82f6;
  --spacing: 16px;
}
.btn {
  background: var(--primary-color);
  padding: var(--spacing);
  transition: transform 0.2s ease-in-out;
}
.btn:hover { transform: scale(1.05); }

@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}
.modal { animation: fadeIn 0.3s ease-in; }
```

6.6 Basic Interview Questions

Q1. What is the CSS Box Model? Every element is a rectangular box composed of, from inside out: **content**, **padding** (space inside the border), **border**, and **margin** (space outside the border). `box-sizing: border-box` makes width/height include padding and border, rather than just content (the `content-box` default).

Q2. What is CSS specificity, and how is it calculated? Specificity determines which rule wins when multiple selectors target the same element. Rough hierarchy (highest to lowest): inline styles > `!important` (overrides everything, use sparingly) > ID selectors > class/attribute/pseudo-class selectors > type/element selectors. When specificity ties, the **later rule in source order** wins.

Q3. What is the difference between `display: none` and `visibility: hidden`? `display: none` removes the element entirely from layout (no space reserved, not accessible). `visibility: hidden` hides the element visually but keeps its space in the layout and it still occupies room.

Q4. What's the difference between relative, absolute, fixed, and sticky positioning? `relative`: positioned relative to its own normal position, still occupies original space. `absolute`: removed from normal flow, positioned relative to nearest positioned ancestor (or viewport). `fixed`: positioned relative to the viewport, stays fixed during scroll. `sticky`: acts like `relative` until a scroll threshold, then acts like `fixed`.

Q5. What is the difference between `em`, `rem`, `%`, and `px`? `px` is an absolute pixel unit. `em` is relative to the *parent* element's font size (compounds with nesting). `rem` is relative to the *root* (`<html>`) font size (predictable, doesn't compound). `%` is relative to the parent's corresponding property (e.g., width relative to parent's width).

Q6. What is the difference between Flexbox and Grid? Flexbox is **one-dimensional** — arranges items along a single row or column, ideal for component-level alignment (navbars, button groups). Grid is **two-dimensional** — arranges items in rows and columns simultaneously, ideal for overall page/layout structure.

6.7 Advanced Interview Questions

Q1. Explain the CSS cascade algorithm in detail. When multiple rules apply to the same element/property, the browser resolves conflicts using, in order: (1) **origin & importance** (user !important > author !important > author normal > user normal > browser default); (2) **specificity** (inline > ID > class/attribute/pseudo-class > element/pseudo-element); (3) **source order** — later rules win on ties. Since CSS4, @layer allows explicit control over cascade layers independent of source order.

Q2. What causes layout reflow, and how do you minimize it? Reflow is triggered by DOM changes, geometry-affecting CSS changes (width, height, margin, font-size), or reading layout properties (offsetWidth) that force synchronous recalculation. Minimize it by: batching DOM/style changes, using transform/opacity for animations (GPU-composited, don't trigger layout), using will-change sparingly to hint the browser, and avoiding reading layout properties immediately after writing styles ("layout thrashing").

Q3. Explain stacking contexts and z-index. z-index only works on **positioned** elements (not static) and operates within a **stacking context** — a self-contained hierarchy of layers. New stacking contexts form due to position + z-index, opacity < 1, transform, filter, or will-change, among others. A z-index: 9999 inside a low-stacking-context parent can still be visually behind a z-index: 1 element in a *higher* stacking context, because comparisons only happen within the same context.

Q4. How does CSS specificity interact with CSS-in-JS/CSS Modules for scoping? CSS Modules and styled-components generate uniquely hashed class names per component at build time, effectively simulating scoped styles without relying on the natural cascade — avoiding global namespace collisions common in large plain-CSS codebases, at the cost of a build step and (for CSS-in-JS) potential runtime overhead versus static CSS Modules.

Q5. What are CSS containment and content-visibility, and why do they matter for performance? contain: layout paint; tells the browser an element's internal layout/paint won't affect the rest of the page, letting the browser skip recalculating outside that boundary. content-visibility: auto goes further — skips rendering work entirely for off-screen content until it's about to become visible, dramatically improving initial render performance for very long pages.

Q6. Explain the difference between CSS transition and animation, and why transform/opacity are preferred for performance. transition interpolates a property between two states triggered by a state change (e.g., :hover). animation (with @keyframes) defines multi-step, potentially looping animations independent of a trigger. Both are preferred to run on transform and opacity specifically because these can be handled by the **compositor thread on the GPU**, skipping layout and paint entirely — animating properties like width or top instead forces expensive reflow on every frame.

7. React

7.1 What is it?

React is a **JavaScript library** (created by Facebook/Meta) for building user interfaces using a **component-based, declarative** model. UI is broken into reusable components that describe *what* the UI should look like for a given state; React efficiently updates the real DOM using a **Virtual DOM diffing** algorithm rather than manual DOM manipulation.

7.2 When is it used?

- **Single Page Applications (SPAs)** with complex, dynamic, interactive UIs.
- **Component-driven design systems** where UI pieces are reused across the app.
- **Dashboards, admin panels, social feeds** requiring frequent state updates.
- Mobile apps via **React Native** (shared logic/patterns with web React).

7.3 How is it used?

1. Build UI as a tree of **components** (functions returning JSX).
2. Manage component-local state with hooks (`useState`), side effects with `useEffect`.
3. On state change, React re-renders the affected component's Virtual DOM representation, **diffs** it against the previous one (reconciliation), and applies only the minimal necessary changes to the real DOM.
4. Compose components via **props** (parent → child data flow) and lift shared state up, or use **Context/Redux** for global/cross-cutting state.
5. Bundle with Vite/Webpack; JSX is transpiled to `React.createElement()` calls via Babel.

7.4 Why is it used?

- **Declarative UI**: describe the end state, not step-by-step DOM mutations — easier to reason about and debug.
- **Component reusability** and composability.
- **Virtual DOM diffing** minimizes expensive real-DOM operations, improving performance.
- **Massive ecosystem** (React Router, Redux, Next.js) and community/job market demand.
- **Unidirectional data flow** makes state changes predictable and easier to trace.

7.5 Core Syntax & Examples

```
import { useState, useEffect } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `Count: ${count}`;
    return () => console.log("cleanup"); // runs before next effect / unmount
  }, [count]); // dependency array

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increment</button>
    </div>
  );
}
```

```

        </div>
    );
}

// Props & composition
function Greeting({ name, children }) {
    return <div><h2>Hello, {name}</h2>{children}</div>;
}

function App() {
    return (
        <Greeting name="Alice">
            <Counter />
        </Greeting>
    );
}

export default App;

```

Custom Hook

```

function useFetch(url) {
    const [data, setData] = useState(null);
    useEffect(() => {
        fetch(url).then(res => res.json()).then(setData);
    }, [url]);
    return data;
}

```

7.6 Basic Interview Questions

Q1. What is JSX? A syntax extension allowing HTML-like markup inside JavaScript, transpiled by Babel into `React.createElement()` calls — it's not understood natively by browsers.

Q2. What is the difference between props and state? Props are read-only data passed from parent to child (external, immutable within the child). State is internal, mutable data managed within a component that triggers re-render when updated.

Q3. What is the Virtual DOM? An in-memory lightweight JS representation of the real DOM. React updates the Virtual DOM first, diffs it against the previous version, and applies only the minimal set of real DOM mutations (reconciliation) — avoiding costly full-page re-renders.

Q4. What are React Hooks? Functions (`useState`, `useEffect`, `useContext`, etc.) that let function components use state and lifecycle-like features without writing class components.

Q5. What is the key prop used for in lists, and why is it important? A unique identifier React uses to track which items in a list changed/added/removed across re-renders, enabling efficient reconciliation. Using array indices as keys can cause bugs when list order changes.

Q6. What is controlled vs uncontrolled components? A controlled component's form value is driven by React state (`value` + `onChange`), making React the single source of truth. An uncontrolled component manages its own internal DOM state, accessed via a `ref` when needed.

7.7 Advanced Interview Questions

Q1. Explain React's reconciliation algorithm and the role of key. React's diffing algorithm compares the new Virtual DOM tree to the previous one using heuristics: (1) elements of different types produce entirely new subtrees; (2) for lists, key lets React match elements across renders by identity rather than position, so it can detect insertions/removals/reorders in $O(n)$ instead of the theoretical $O(n^3)$ tree-diff, avoiding unnecessary unmount/remount of unchanged items.

Q2. Explain the differences between useMemo, useCallback, and React.memo. useMemo memoizes a **computed value**, recomputing only when dependencies change. useCallback memoizes a **function reference** (equivalent to `useMemo(() => fn, deps)`), useful to prevent unnecessary re-renders of children receiving that function as a prop. React.memo wraps a component to skip re-rendering if its props are shallowly equal to the previous render.

Q3. What is the Context API, and what are its performance pitfalls? Context provides a way to pass data through the component tree without manually threading props at every level ("prop drilling"). Pitfall: any component consuming a Context re-renders whenever the Context value changes, even if it only cares about part of it — for high-frequency updates across large trees, dedicated state managers (Redux, Zustand) or splitting contexts is often preferred.

Q4. Explain the React Fiber architecture. Fiber (React 16+) is a reimplement of React's core reconciliation algorithm as an **incremental, interruptible** unit-of-work tree, replacing the old synchronous recursive stack-based reconciler. It allows React to pause, abort, or prioritize rendering work (e.g., yielding to the browser for high-priority user input) — enabling features like Concurrent Mode, Suspense, and time-slicing.

Q5. What is Suspense and how does it relate to data fetching / code-splitting? `<Suspense fallback={...}>` lets a component "wait" for something (a lazily-loaded component via `React.lazy`, or async data source integrated with Suspense) before rendering, showing a fallback UI in the meantime — declaratively handling loading states without manual boolean flags scattered across components.

Q6. Explain the Rules of Hooks and why they exist. Hooks must be called (1) only at the top level of a function component (never inside loops/conditions/nested functions) and (2) only from React function components or custom hooks. This is because React tracks hooks by **call order** per component instance (an internal linked list), not by name — conditional hook calls would misalign state between renders.

Q7. How does React Server Components (RSC) differ from traditional SSR? Traditional SSR renders the whole component tree to HTML on the server for the initial load, then **hydrates** it entirely on the client (re-executing all component code client-side). RSC allows *specific components* to run **only** on the server (zero client-side JS bundle for those components, direct backend/database access), streaming their rendered output while client components hydrate separately — reducing bundle size and improving performance for data-heavy pages (used in Next.js App Router).

8. Angular

8.1 What is it?

Angular is a complete, opinionated **TypeScript-based front-end framework** (by Google) for building large-scale SPAs. Unlike React (a UI library), Angular is a full platform — including routing, forms, HTTP client, dependency injection, and CLI tooling — out of the box.

8.2 When is it used?

- **Large enterprise applications** that benefit from Angular's opinionated structure and consistency across large teams.
- Applications needing **built-in, batteries-included tooling** (forms validation, HTTP interceptors, routing guards) without assembling many third-party libraries.
- Teams already using **TypeScript** who want strong typing enforced framework-wide.

8.3 How is it used?

1. Scaffold a project using the **Angular CLI** (`ng new`, `ng generate component`).
2. Build the app as a tree of **components** (each with a TypeScript class, an HTML template, and CSS), organized into **modules** (`NgModule`) (or standalone components in modern Angular).
3. Use **two-way data binding** (`[(ngModel)]`), **directives** (`*ngIf`, `*ngFor`), and **dependency injection** for services (e.g., HTTP data services shared across components).
4. Angular's compiler (Ivy) compiles templates **ahead-of-time (AOT)** into efficient JS for production.
5. Use RxJS **Observables** for handling asynchronous streams (HTTP calls, events).

8.4 Why is it used?

- **All-in-one framework**: routing, forms, HTTP, testing utilities included — less decision fatigue than assembling a React stack.
- **TypeScript-first**, enforcing type safety across the whole application.
- **Dependency Injection** built into the framework promotes testable, modular, loosely-coupled code.
- **Two-way data binding** simplifies form-heavy applications.
- Strong conventions make large teams/codebases more consistent and maintainable.

8.5 Core Syntax & Examples

Component

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-counter',
  template: `
    <p>Count: {{ count }}</p>
    <button (click)="increment()">Increment</button>
  `
})
export class CounterComponent {
  count = 0;
```

```
increment() { this.count++; }
}
```

Data Binding & Directives

```
<!-- Interpolation -->
<h1>{{ title }}</h1>

<!-- Property binding -->
<img [src]="imageUrl">

<!-- Event binding -->
<button (click)="onClick()">Click</button>

<!-- Two-way binding -->
<input [(ngModel)]="name">

<!-- Structural directives -->
<div *ngIf="isVisible">Visible content</div>
<li *ngFor="let item of items">{{ item.name }}</li>
```

Service & Dependency Injection

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';

@Injectable({ providedIn: 'root' })
export class UserService {
  constructor(private http: HttpClient) {}
  getUsers() {
    return this.http.get('/api/users');
  }
}

@Component({ selector: 'app-users', template: `...` })
export class UsersComponent {
  constructor(private userService: UserService) {} // DI injects it automatically
}
```

8.6 Basic Interview Questions

Q1. What is Angular, and how does it differ from React? Angular is a full, opinionated framework (routing, DI, forms, HTTP built-in), using TypeScript and a component + template model with two-way binding. React is a UI-rendering library only, more flexible/unopinionated, requiring you to choose additional libraries for routing/state.

Q2. What are components and modules in Angular? A component controls a section of the UI (template + logic + styles). A module (NgModule) groups related components, directives, and services into a cohesive functional unit; modern Angular also supports **standalone components** that don't require NgModule.

Q3. What is data binding, and what types exist? Connecting the component's data to

the template. Types: **interpolation** (`{{ }}`, one-way component→view), **property binding** (`[prop]`, one-way), **event binding** (`((event))`, view→component), and **two-way binding** (`[(ngModel)]`), combines both).

Q4. What is dependency injection (DI) in Angular? A design pattern where a class receives its dependencies (like services) from an external injector rather than creating them itself, promoted by Angular’s built-in **hierarchical injector** system, improving testability and modularity.

Q5. What is the difference between `*ngIf` and `*ngFor`? `*ngIf` conditionally includes/excludes an element from the DOM based on a boolean expression. `*ngFor` repeats an element for each item in an iterable (array), similar to `.map()` in React/JS.

8.7 Advanced Interview Questions

Q1. Explain Angular’s change detection mechanism and `zone.js`. Angular wraps async browser APIs (setTimeout, DOM events, XHR) using **zone.js**, which lets Angular know *when* something might have changed, triggering **change detection**: Angular walks the component tree, checks each component’s bindings, and updates the DOM if template expressions changed. By default this is checked for the entire tree (“default” strategy); `ChangeDetectionStrategy.OnPush` restricts checks to only run when a component’s `@Input()` reference changes (or an event fires within it) — a major performance optimization for large trees.

Q2. Explain Angular’s Ivy renderer and AOT vs JIT compilation. **Ivy** (default since Angular 9) is Angular’s rendering/compilation engine, generating more efficient, tree-shakeable instructions per component. **AOT (Ahead-of-Time)** compilation converts templates to JS during the build step (smaller bundles, faster startup, catches template errors at build time) — the default for production. **JIT (Just-in-Time)** compiles templates in the browser at runtime — mainly used in older dev workflows.

Q3. Explain RxJS Observables and common operators used in Angular. Observables represent asynchronous streams of data over time (unlike Promises which resolve once). Angular uses them extensively for HTTP calls, forms, and events. Common operators: `map` (transform emitted values), `filter`, `switchMap` (cancel previous inner observable when a new source value arrives — critical for typeahead search to avoid race conditions), `debounceTime`, `catchError`, and `combineLatest`.

Q4. What are Angular Guards and Resolvers? Guards (`CanActivate`, `CanDeactivate`) control route access — e.g., blocking navigation to a route unless the user is authenticated. **Resolvers** pre-fetch data before a route activates, so the component has data immediately available on render, avoiding loading-state flicker.

Q5. Explain the Angular hierarchical dependency injection system. Angular’s injector forms a **tree mirroring the component tree**. A service registered `providedIn: 'root'` is a true app-wide singleton. A service provided at a specific component level creates a **new instance scoped to that component subtree**, shadowing any parent-level provider — enabling scenarios like per-feature-module or per-component service instances (e.g., isolated state per dynamically created dialog).

Q6. What is the difference between NgModule-based and standalone component architecture (Angular 14+)? Standalone components declare their own dependencies (imports) directly via the `standalone: true + imports: []` metadata, eliminating the need for NgModule boilerplate for many apps — simplifying the mental model, improving tree-shaking, and making lazy-loading individual components more straightforward.

9. Streamlit

9.1 What is it?

Streamlit is a **Python framework for rapidly building interactive data/ML web apps** without needing front-end skills (no HTML/CSS/JS required). You write a normal top-to-bottom Python script using Streamlit's API, and it renders as a live, interactive web application.

9.2 When is it used?

- **ML/data science demos and prototypes** — showcasing a trained model to stakeholders quickly.
- **Internal dashboards and data exploration tools** for teams.
- **Rapid MVP creation** where speed matters more than deep UI customization.
- Turning a Jupyter notebook analysis into a **shareable interactive app**.

9.3 How is it used?

1. Write a .py script using `st.*` functions to render widgets/outputs.
2. Run with `streamlit run app.py` — it starts a local web server and opens the app in a browser.
3. Streamlit **re-runs the entire script top-to-bottom** on every user interaction (button click, slider change), using its internal diffing to efficiently update only the changed UI elements.
4. Use `st.session_state` to persist values across these re-runs (since local variables reset each time).
5. Use `@st.cache_data` / `@st.cache_resource` to avoid re-computing expensive operations (e.g., loading a large model or dataset) on every re-run.

9.4 Why is it used?

- **Extremely fast to build** — a full interactive app in a few dozen lines of pure Python, no separate front-end code.
- Ideal for **data scientists/ML engineers** who don't want to learn a JS framework just to demo a model.
- **Built-in widgets** for common data app needs: sliders, file uploaders, charts, dataframes.
- **Easy deployment** via Streamlit Community Cloud or standard Python hosting.

9.5 Core Syntax & Examples

```
import streamlit as st
import pandas as pd

st.title("Simple Data App")
st.write("Upload a CSV to explore it interactively.")

uploaded_file = st.file_uploader("Choose a CSV file", type="csv")

if uploaded_file is not None:
    df = pd.read_csv(uploaded_file)
    st.dataframe(df.head())
```

```

column = st.selectbox("Select column to plot", df.columns)
st.line_chart(df[column])

# Widgets & layout
name = st.text_input("Enter your name")
age = st.slider("Select age", 0, 100, 25)

col1, col2 = st.columns(2)
with col1:
    st.metric("Age", age)
with col2:
    st.metric("Name length", len(name))

# Caching expensive computation
@st.cache_data
def load_large_dataset():
    return pd.read_csv("large_file.csv")

# Session state (persists across re-runs)
if "count" not in st.session_state:
    st.session_state.count = 0

if st.button("Increment"):
    st.session_state.count += 1

st.write(f"Count: {st.session_state.count}")

```

9.6 Basic Interview Questions

Q1. What is Streamlit and what problem does it solve? An open-source Python framework that lets developers build interactive web apps for data/ML projects using pure Python scripts, eliminating the need for separate front-end (HTML/CSS/JS) development.

Q2. How does Streamlit's execution model work? Streamlit re-runs the **entire script from top to bottom** every time the user interacts with a widget (e.g., moves a slider), rather than using an event-callback model like traditional web frameworks — Streamlit then intelligently diffs and updates only the parts of the UI that changed.

Q3. What are widgets in Streamlit? Give examples. Interactive UI elements like `st.button()`, `st.slider()`, `st.selectbox()`, `st.text_input()`, `st.file_uploader()` — each returns the user's current input value directly in the script's control flow.

Q4. How do you display data (tables/charts) in Streamlit? `st.dataframe()` / `st.table()` for tabular data, and built-in chart helpers `st.line_chart()`, `st.bar_chart()`, `st.area_chart()`, or `st.pyplot()/st.plotly_chart()` for custom Matplotlib/Plotly figures.

Q5. How do you run and deploy a Streamlit app? Run locally with `streamlit run app.py`. Deploy via Streamlit Community Cloud (connects to a GitHub repo) or any standard Python hosting (Docker container running the same command, exposing the default port 8501).

9.7 Advanced Interview Questions

Q1. Why is `st.session_state` necessary, and what problem does it solve? Because Streamlit re-executes the whole script on every interaction, regular Python variables are **re-set to their initial values on every re-run**. `st.session_state` is a dict-like persistent store

tied to the user's browser session, letting you maintain state (counters, form data, computed results) across re-runs.

Q2. Explain Streamlit's caching decorators — `st.cache_data` vs `st.cache_resource`, and why they matter. `@st.cache_data` caches the **return value** of a function based on its input arguments (hashed) — ideal for data-loading/transformation functions that return serializable data (DataFrames, arrays); Streamlit stores a copy to prevent accidental mutation of the cached object. `@st.cache_resource` caches **non-serializable, shared resources** (ML models, DB connections) as a singleton reference (not copied) across sessions/re-runs. Without caching, expensive operations (loading a multi-GB model) would re-run on every single widget interaction, making the app unusably slow.

Q3. How does Streamlit handle multi-page apps and app architecture at scale? Via a pages/ directory convention (each .py file becomes a separate page in the sidebar navigation) or the newer `st.navigation/st.Page` API for programmatic multi-page routing — useful for organizing larger internal tools beyond a single script.

Q4. What are the performance/scalability limitations of Streamlit compared to a React/Angular front-end, and when would you NOT choose it? Since the whole script re-runs on each interaction, very complex apps with many interdependent widgets can become slow without careful caching/state management. It also offers less fine-grained UI/UX customization than a dedicated front-end framework, and isn't designed for large numbers of concurrent users on a single server instance without additional scaling infrastructure. It's best suited for internal tools/prototypes/demos, not consumer-facing, highly customized production web apps.

Q5. How would you integrate a trained ML model into a Streamlit app for live inference? Load the model once using `@st.cache_resource` (avoiding reloading it on every re-run), collect user inputs via widgets, run `model.predict()` on submission (often behind an `st.form()` to batch inputs and avoid re-running on every keystroke/slider move), and display results with `st.write()/charts` — a common pattern for showcasing scikit-learn or TensorFlow models.

10. Scikit-learn

10.1 What is it?

Scikit-learn (sklearn) is an open-source **Python machine learning library** built on NumPy, SciPy, and matplotlib, providing simple, consistent APIs for classical ML: classification, regression, clustering, dimensionality reduction, model selection, and preprocessing.

10.2 When is it used?

- **Classical/traditional ML tasks** on structured/tabular data: regression, classification, clustering — not deep learning (use TensorFlow/PyTorch for that).
- **Rapid prototyping** of ML pipelines and baseline models.
- **Feature engineering & preprocessing** (scaling, encoding, imputation).
- **Model evaluation & hyperparameter tuning** (cross-validation, grid search).
- Educational/production use where interpretable, well-tested classical algorithms suffice (e.g., fraud detection, churn prediction, credit scoring).

10.3 How is it used?

1. Prepare data as NumPy arrays or pandas DataFrames.
2. Split into train/test sets (`train_test_split`).
3. Preprocess features (`StandardScaler`, `OneHotEncoder`) — fit on training data only, transform both.
4. Instantiate an **estimator** (model), call `.fit(X_train, y_train)`.
5. Predict with `.predict(X_test)`, evaluate with metrics (`accuracy_score`, `mean_squared_error`).
6. Tune hyperparameters with `GridSearchCV`/`RandomizedSearchCV` using cross-validation.
7. Chain steps into a Pipeline for reproducibility and to prevent data leakage.

10.4 Why is it used?

- **Consistent, unified API:** every estimator follows the same `.fit()` / `.predict()` / `.transform()` convention, making algorithms interchangeable.
- **Comprehensive:** covers most classical ML algorithms and preprocessing needs in one library.
- **Well-documented, well-tested, production-grade** implementations.
- **Interoperates seamlessly** with NumPy/pandas/matplotlib — the standard Python data science stack.
- Built-in tools for **model selection, validation, and pipeline construction** reduce boilerplate and bugs like data leakage.

10.5 Core Syntax & Examples

```
import numpy as np
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Load / split data
X, y = load_dataset() # X: features, y: labels
X_train, X_test, y_train, y_test = train_test_split(
```

```

    X, y, test_size=0.2, random_state=42, stratify=y
)

# 2. Build a pipeline (preprocessing + model, prevents leakage)
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", RandomForestClassifier(random_state=42))
])

# 3. Hyperparameter tuning with cross-validation
param_grid = {"clf__n_estimators": [100, 200], "clf__max_depth": [5, 10, None]}
grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring="accuracy")
grid_search.fit(X_train, y_train)

print("Best params:", grid_search.best_params_)

# 4. Evaluate on held-out test set
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))

```

Common estimators

```

from sklearn.linear_model import LogisticRegression, LinearRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

# Unsupervised example
kmeans = KMeans(n_clusters=3, random_state=42).fit(X)
labels = kmeans.labels_

pca = PCA(n_components=2)
X_reduced = pca.fit_transform(X)

```

10.6 Basic Interview Questions

Q1. What is scikit-learn used for? Classical machine learning in Python — classification, regression, clustering, dimensionality reduction, model selection, and data preprocessing — with a consistent, easy-to-use API.

Q2. What is the difference between `.fit()`, `.transform()`, and `.fit_transform()`? `.fit()` learns parameters from data (e.g., mean/std for a scaler, or model weights for an estimator). `.transform()` applies a previously learned transformation to data. `.fit_transform()` does both in one call (used on training data); on test data you should only call `.transform()` using parameters learned from training, to avoid data leakage.

Q3. Why do we split data into train and test sets? To evaluate how well a model generalizes to **unseen** data, rather than just measuring how well it memorized the training data — protecting against overestimating real-world performance.

Q4. What is overfitting and underfitting? Overfitting: the model learns noise/details

specific to the training data and performs poorly on new data (high variance). Underfitting: the model is too simple to capture the underlying pattern, performing poorly on both training and test data (high bias).

Q5. What is cross-validation, and why use it over a single train/test split? Cross-validation (commonly **k-fold**) splits the data into k parts, trains on $k-1$ folds and validates on the remaining fold, repeating k times and averaging results — giving a more robust estimate of model performance than a single train/test split, especially with limited data.

Q6. Name common evaluation metrics for classification vs regression. Classification: accuracy, precision, recall, F1-score, ROC-AUC, confusion matrix. Regression: Mean Squared Error (MSE), Root MSE, Mean Absolute Error (MAE), R^2 score.

10.7 Advanced Interview Questions

Q1. Explain the bias-variance tradeoff and how model complexity relates to it. **Bias** is error from overly simplistic assumptions (underfitting); **variance** is error from sensitivity to fluctuations in training data (overfitting). Increasing model complexity (e.g., deeper trees, more polynomial features) typically decreases bias but increases variance. The goal is to find the complexity level that minimizes **total** expected error ($\text{bias}^2 + \text{variance} + \text{irreducible noise}$), often via cross-validation and regularization.

Q2. Why is it critical to use Pipeline rather than manually preprocessing before splitting? If you fit a scaler/encoder on the **full dataset before** splitting into train/test, statistics from the test set (mean, std, category frequencies) leak into training — inflating validation performance unrealistically (“data leakage”). Pipeline combined with Grid-SearchCV/cross_val_score ensures preprocessing is **refit independently within each cross-validation fold**, using only that fold’s training data — giving an honest performance estimate.

Q3. Explain regularization (L1 vs L2) and why it’s used. Regularization adds a penalty term to the loss function to discourage overly large model weights, reducing overfitting. **L1 (Lasso)** adds the sum of absolute weight values — tends to push some coefficients exactly to zero, performing implicit **feature selection**. **L2 (Ridge)** adds the sum of squared weights — shrinks all coefficients smoothly toward zero without eliminating them, handling multicollinearity well. ElasticNet combines both.

Q4. How do ensemble methods like Random Forest and Gradient Boosting differ? **Random Forest** (bagging): trains many decision trees independently and in parallel on bootstrapped samples with random feature subsets, then averages/votes their predictions — reduces variance, robust to overfitting, easily parallelizable. **Gradient Boosting** (e.g., GradientBoostingClassifier, XGBoost/LightGBM): trains trees **sequentially**, each new tree correcting the residual errors of the previous ensemble — typically achieves higher accuracy but is more prone to overfitting and is inherently sequential (harder to parallelize across trees).

Q5. How do you handle class imbalance in a classification problem using scikit-learn? Techniques include: using `class_weight="balanced"` (available on many estimators) to penalize misclassifying the minority class more heavily; resampling (oversampling the minority class e.g. via SMOTE from `imbalanced-learn`, or undersampling the majority); choosing appropriate metrics (precision/recall/F1/PR-AUC instead of raw accuracy, which is misleading on imbalanced data); and stratified k -fold cross-validation (`StratifiedKFold`) to preserve class ratios in each fold.

Q6. What is the purpose of random_state, and why does reproducibility matter in ML experiments? Many scikit-learn operations involve randomness (train/test splitting, weight initialization, bootstrap sampling in Random Forest). Fixing `random_state` to a constant seed

ensures the same random choices are made every run, making results **reproducible** — essential for debugging, fair model comparison, and scientific rigor.

Q7. Explain feature scaling — when is it necessary and when isn't it? Distance-based and gradient-based algorithms (KNN, SVM, logistic/linear regression, neural networks, PCA) are sensitive to feature scale — a feature with a larger numeric range can dominate the distance/gradient calculations, so `StandardScaler`/`MinMaxScaler` is necessary. Tree-based models (Decision Trees, Random Forest, Gradient Boosting) split on threshold comparisons per feature independently and are **scale-invariant**, so scaling is not required for them.

11. TensorFlow.js

11.1 What is it?

TensorFlow.js (TFjs) is a **JavaScript library for training and running machine learning / deep learning models directly in the browser or in Node.js**, using WebGL/WebGPU/WASM for GPU/CPU acceleration client-side, without a server round-trip.

11.2 When is it used?

- **In-browser ML inference** — running a pre-trained model directly on the client (e.g., real-time pose detection, face detection, object detection in a webcam feed).
- **Privacy-sensitive applications** — data never leaves the user's device (on-device inference).
- **Interactive ML demos/education** — visualizing model training live in a web page.
- **Converting existing Python-trained models** (Keras/TensorFlow SavedModel) to run in JS via the TFjs converter.
- **Transfer learning in the browser** — fine-tuning a pre-trained model on user-provided data client-side.

11.3 How is it used?

1. Include TFjs via `<script>` CDN tag or `npm install @tensorflow/tfjs`.
2. Either **load a pre-trained model** (`tf.loadLayersModel()` for Keras-converted models, or use pre-built models from `@tensorflow-models/*` like `mobilenet`, `posenet`), or **build/train a model from scratch** using the Layers API (similar to Keras).
3. Prepare input data as **Tensors** (`tf.tensor()`), run inference (`model.predict()`).
4. For training, define a model, compile with an optimizer/loss, and call `.fit()` — computation runs on GPU via WebGL backend when available.
5. Dispose of tensors manually (`tensor.dispose()`) or use `tf.tidy()` to prevent memory leaks, since JS doesn't automatically garbage-collect GPU-backed tensor memory.

11.4 Why is it used?

- **Zero-install, cross-platform inference** — runs anywhere a browser runs, no server needed for prediction.
- **Low latency** — no network round-trip to a backend for each prediction.
- **Privacy**: sensitive data (camera/mic feed) can be processed entirely on-device.
- **Reduced server costs** — offloads inference compute to the client.
- **Interoperability** with the Python TensorFlow/Keras ecosystem via model conversion.

11.5 Core Syntax & Examples

```
import * as tf from '@tensorflow/tfjs';

// Tensors
const a = tf.tensor2d([[1, 2], [3, 4]]);
const b = tf.tensor2d([[5, 6], [7, 8]]);
const sum = a.add(b);
sum.print();

// Building a simple sequential model (Layers API)
const model = tf.sequential();
```

```

model.add(tf.layers.dense({ units: 16, activation: 'relu', inputShape: [4] }));
model.add(tf.layers.dense({ units: 1, activation: 'sigmoid' }));

model.compile({
  optimizer: tf.train.adam(0.01),
  loss: 'binaryCrossentropy',
  metrics: ['accuracy']
});

// Training
const xs = tf.tensor2d([[0,0,1,1],[1,1,0,0]]);
const ys = tf.tensor2d([[0],[1]]);

await model.fit(xs, ys, {
  epochs: 50,
  callbacks: {
    onEpochEnd: (epoch, logs) => console.log(`Epoch ${epoch}: loss=${logs.loss}`)
  }
});

// Inference
const prediction = model.predict(tf.tensor2d([[1,0,1,0]]));
prediction.print();

// Memory management
tf.tidy(() => {
  const t = tf.tensor([1,2,3]);
  return t.square(); // intermediate tensors auto-disposed
});

// Loading a pre-trained model
const preTrained = await tf.loadLayersModel('https://example.com/model.json');

```

11.6 Basic Interview Questions

Q1. What is TensorFlow.js and how does it differ from Python TensorFlow? TF.js runs machine learning models in JavaScript, in the browser (using WebGL/WebGPU for GPU acceleration) or in Node.js, whereas Python TensorFlow runs on servers/workstations. TF.js is primarily used for **client-side inference and lightweight training**, while Python TensorFlow is used for heavy-duty, large-scale model training.

Q2. What is a Tensor in TensorFlow.js? A multi-dimensional array (generalization of scalars, vectors, matrices) that is the fundamental data structure for all computations — created via `tf.tensor()`, `tf.tensor2d()`, etc.

Q3. What are the two main APIs TF.js provides for building models? The **Layers API** (high-level, Keras-like — `tf.sequential()`, `tf.layers.dense()`) for common model architectures, and the lower-level **Core/Ops API** (`tf.matMul`, `tf.add`) for custom, fine-grained tensor operations.

Q4. How do you load a pre-trained model in TF.js? `tf.loadLayersModel(url)` for Keras-style models converted via the `tensorflowjs_converter` tool, or `tf.loadGraphModel(url)` for models exported as TensorFlow SavedModel/frozen graphs.

Q5. What backends does TF.js support, and why does the backend matter? CPU

(plain JS, slowest), WebGL (GPU-accelerated via browser's WebGL, most common), WASM (good CPU performance for smaller models), and WebGPU (newer, faster GPU backend). The backend affects both inference speed and numerical precision (e.g., WebGL uses 32-bit floats but internally may have precision quirks on some devices).

11.7 Advanced Interview Questions

Q1. Why is manual memory management (`tf.tidy()`, `.dispose()`) necessary in TF.js, unlike typical JS objects? Tensors, especially on the WebGL backend, are backed by **GPU memory (textures)**, which JavaScript's garbage collector does not manage — the GC only tracks the small JS wrapper object referencing the underlying GPU buffer. If tensors aren't explicitly disposed, GPU memory leaks and can crash the browser tab. `tf.tidy()` automatically disposes all intermediate tensors created within its callback except the one(s) returned, making cleanup declarative and less error-prone than manual `.dispose()` calls everywhere.

Q2. Explain the difference between the WebGL and WASM backends, and when you'd choose one over the other. **WebGL** encodes tensor operations as GPU shader programs — excellent throughput for large models/batches but has per-operation dispatch overhead and browser/driver variability. **WASM** (WebAssembly, optionally with SIMD) runs on CPU — often faster than WebGL for very small models or low-batch inference (e.g., detecting a single face per frame) because it avoids GPU dispatch overhead and works consistently across devices without GPU driver quirks, including on lower-end devices.

Q3. How does TF.js enable transfer learning directly in the browser? By loading a pre-trained base model (e.g., MobileNet), **freezing** most of its layers (setting `trainable = false`), replacing the final classification layer(s) with new, untrained layers matching the target task's number of classes, and then calling `.fit()` on a small user-provided dataset (e.g., captured via webcam) — only the new layers' weights update, making training fast enough to run interactively client-side.

Q4. What are the performance/security tradeoffs of client-side ML inference vs server-side inference? Client-side: lower latency (no network round trip), better privacy (data stays local), reduced server cost, but the **model itself is exposed** to the client (can be extracted/reverse-engineered from downloaded weights), inference speed varies wildly across user devices, and larger models increase page load time. Server-side: consistent performance/hardware control, model IP stays protected, but incurs network latency, ongoing server/GPU costs, and raises privacy considerations for sensitive input data being transmitted.

Q5. How would you convert an existing Keras/TensorFlow Python model to run in TF.js? Use the `tensorflowjs_converter` CLI tool (from the `tensorflowjs` pip package) to convert a SavedModel or Keras `.h5` file into TF.js's `model.json` + sharded binary weight files format, then load it in the browser with `tf.loadLayersModel()` (for Keras-derived models) or `tf.loadGraphModel()` (for SavedModel/frozen-graph-derived models exported for inference-only use, which is typically faster since it skips training-only ops).

12. MySQL

12.1 What is it?

MySQL is an open-source **Relational Database Management System (RDBMS)** that stores data in structured tables with rows and columns, using **SQL (Structured Query Language)** to define, query, and manipulate data. It enforces a fixed schema and supports **ACID-compliant transactions** (with the InnoDB storage engine).

12.2 When is it used?

- **Web application back-ends** — one of the most popular DBs for web apps (part of the classic LAMP stack), e-commerce, CMS platforms (WordPress).
- **Applications requiring structured, relational data** with well-defined relationships (foreign keys) between entities.
- **Transactional systems** needing ACID guarantees — banking-adjacent apps, order/inventory management.
- **Read-heavy applications** — MySQL is well optimized for read-scaling via replication.

12.3 How is it used?

1. Design a **schema**: tables, columns with data types, primary keys, foreign keys, indexes.
2. Interact via **SQL**: CREATE TABLE, INSERT, SELECT, UPDATE, DELETE.
3. Use **joins** to combine data across related tables.
4. Wrap multiple related writes in a **transaction** (BEGIN, COMMIT, ROLLBACK) to maintain consistency.
5. Add **indexes** on frequently queried columns to speed up lookups.
6. Connect from an application via a database driver/ORM (JDBC for Java, mysql2 for Node, SQLAlchemy for Python).

12.4 Why is it used?

- **ACID compliance** ensures data integrity for critical transactional data.
- **Mature, well-documented, widely supported** — huge community, tooling, and hosting support.
- **Strong relational integrity** via foreign keys, constraints, and normalization.
- **Good performance** for read-heavy and moderately write-heavy workloads, with mature replication support.
- **Free and open-source**, with enterprise support options available.

12.5 Core Syntax & Examples

```
-- Create table
CREATE TABLE employees (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  department_id INT,
  salary DECIMAL(10,2),
  hire_date DATE,
  FOREIGN KEY (department_id) REFERENCES departments(id)
);

-- Insert
```

```

INSERT INTO employees (name, department_id, salary, hire_date)
VALUES ('Alice', 1, 65000.00, '2023-01-15');

-- Select with filtering, sorting
SELECT name, salary FROM employees
WHERE salary > 50000
ORDER BY salary DESC
LIMIT 10;

-- Joins
SELECT e.name, d.department_name
FROM employees e
INNER JOIN departments d ON e.department_id = d.id;

-- Aggregation
SELECT department_id, AVG(salary) AS avg_salary, COUNT(*) AS emp_count
FROM employees
GROUP BY department_id
HAVING AVG(salary) > 60000;

-- Update / Delete
UPDATE employees SET salary = salary * 1.1 WHERE department_id = 2;
DELETE FROM employees WHERE hire_date < '2015-01-01';

-- Transactions
START TRANSACTION;
UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;
COMMIT; -- or ROLLBACK on error

-- Index
CREATE INDEX idx_salary ON employees(salary);

-- Subquery
SELECT name FROM employees
WHERE department_id IN (SELECT id FROM departments WHERE location = 'NY');

```

12.6 Basic Interview Questions

Q1. What is the difference between DELETE, TRUNCATE, and DROP? DELETE removes specific rows (with optional WHERE), is logged row-by-row, and can be rolled back within a transaction. TRUNCATE removes **all** rows quickly (resets auto-increment), is minimally logged, and generally can't be rolled back in MySQL/InnoDB in the same way. DROP removes the entire table structure and data permanently.

Q2. What is a primary key vs a foreign key? A primary key uniquely identifies each row in a table (cannot be NULL, must be unique). A foreign key is a column referencing another table's primary key, enforcing referential integrity between related tables.

Q3. What are the different types of JOINS? INNER JOIN (only matching rows in both tables), LEFT JOIN (all rows from left table + matches from right, NULL if no match), RIGHT JOIN (all rows from right table + matches from left), FULL OUTER JOIN (all rows from both, MySQL emulates this via UNION of LEFT and RIGHT JOIN since it lacks native support).

Q4. What is normalization, and name the normal forms. The process of organizing data to reduce redundancy and improve integrity. **1NF**: atomic column values, no repeating groups. **2NF**: 1NF + no partial dependency on part of a composite key. **3NF**: 2NF + no transitive dependency (non-key columns depend only on the key).

Q5. What is an index, and why does it improve query performance? A data structure (typically a **B-Tree** in InnoDB) that maintains a sorted reference to table rows for a given column, allowing the query engine to locate matching rows in $O(\log n)$ instead of scanning the entire table ($O(n)$) — at the cost of extra storage and slower writes (since indexes must also be updated).

Q6. What is the difference between WHERE and HAVING? WHERE filters individual rows **before** grouping/aggregation. HAVING filters **groups** after GROUP BY/aggregation has been applied — e.g., filtering on an aggregate like `AVG(salary)`.

12.7 Advanced Interview Questions

Q1. Explain ACID properties in the context of MySQL transactions. **Atomicity**: a transaction's operations either all succeed or all fail together (via COMMIT/ROLLBACK). **Consistency**: a transaction moves the database from one valid state to another, respecting all constraints. **Isolation**: concurrent transactions don't interfere with each other's intermediate states (governed by isolation levels). **Durability**: once committed, changes survive system crashes (via the InnoDB redo log / write-ahead logging).

Q2. Explain MySQL's transaction isolation levels and their trade-offs. From lowest to highest isolation: **READ UNCOMMITTED** (allows dirty reads), **READ COMMITTED** (no dirty reads, but non-repeatable reads possible), **REPEATABLE READ** (MySQL/InnoDB's default — prevents dirty and non-repeatable reads via MVCC snapshots, but phantom reads are largely mitigated too via gap locking), and **SERIALIZABLE** (strictest, effectively locks reads, preventing all anomalies at the cost of concurrency/throughput). Higher isolation reduces anomalies but increases locking/contention.

Q3. How does InnoDB's B-Tree indexing and clustered index work? InnoDB stores table data physically ordered by the **primary key**, called the **clustered index** — the leaf nodes of the primary key's B-Tree contain the actual row data. **Secondary indexes** store the indexed column(s) plus a reference to the primary key value (not the row's physical location), so looking up via a secondary index requires an extra lookup into the clustered index ("bookmark lookup") to fetch full row data unless it's a **covering index** (all needed columns are in the secondary index itself).

Q4. What is the N+1 query problem, and how do you avoid it? It occurs when fetching a list of N parent records, then executing a separate query for each record's related child data ($1 + N$ queries total) instead of one efficient query. Avoided using JOINS to fetch related data in a single query, or ORM eager-loading features (e.g., batch-fetching related entities).

Q5. Explain deadlocks in MySQL and how InnoDB handles them. A deadlock occurs when two or more transactions each hold a lock the other needs, forming a cycle of waiting. InnoDB's deadlock detector periodically checks the "waits-for" graph and automatically **rolls back one of the transactions** (typically the one with the smaller estimated cost) to break the cycle, returning an error to that transaction's client, which should retry. Application-level mitigation includes acquiring locks/updating rows in a **consistent order** across transactions.

Q6. What is query execution plan analysis (EXPLAIN), and what should you look for? `EXPLAIN SELECT ...` shows how MySQL's optimizer plans to execute a query — the type column (const/ref/range are good, ALL means a full table scan — bad for large tables), key (which index is used, if any), rows (estimated rows examined), and Extra (e.g., "Using filesort")

or “Using temporary” indicate potentially expensive operations). It’s the primary tool for diagnosing slow queries and deciding where to add indexes.

Q7. Explain replication in MySQL and common topologies. MySQL replication copies data from a **primary (master)** to one or more **replicas (slaves)**, traditionally via the primary’s **binary log (binlog)** being streamed and replayed on replicas (statement-based, row-based, or mixed replication). Common uses: **read scaling** (route read queries to replicas), **high availability/failover**, and **backups without impacting the primary**. Modern setups often use **semi-synchronous** or **group replication** for stronger consistency guarantees than fully asynchronous replication.

13. Oracle Database

13.1 What is it?

Oracle Database is a commercial, enterprise-grade **object-relational database management system (ORDBMS)** developed by Oracle Corporation, known for high scalability, robustness, advanced features (partitioning, advanced replication, PL/SQL procedural extensions), and heavy use in large enterprises (banking, telecom, government).

13.2 When is it used?

- **Large-scale enterprise systems** requiring high availability, strong security, and vendor support/SLAs (banking, ERP systems like SAP, telecom billing).
- Applications needing **advanced procedural SQL logic** on the database side via **PL/SQL** (stored procedures, triggers, packages).
- Systems requiring Oracle-specific advanced features: **partitioning** for very large tables, **RAC (Real Application Clusters)** for high availability/scale-out, **Data Guard** for disaster recovery.
- Legacy enterprise systems already standardized on Oracle.

13.3 How is it used?

1. Design schemas using SQL DDL, similar to standard relational databases, with Oracle-specific data types (VARCHAR2, NUMBER, CLOB).
2. Write business logic server-side using **PL/SQL** — procedures, functions, packages, triggers — executed inside the database engine itself for performance and centralized logic.
3. Use **sequences** (CREATE SEQUENCE) for auto-incrementing IDs (Oracle historically lacked native AUTO_INCREMENT until IDENTITY columns were added in 12c).
4. Manage concurrency and consistency via Oracle's **multi-version concurrency control (MVCC)** using undo segments.
5. Use Oracle-specific tools: ***SQL*Plus, SQL Developer, Oracle Enterprise Manager*** for administration and tuning.

13.4 Why is it used?

- **Enterprise-grade reliability, scalability, and security** features (fine-grained auditing, Transparent Data Encryption, Virtual Private Database).
- **PL/SQL** enables powerful, high-performance server-side business logic close to the data.
- **High availability features**: RAC (active-active clustering), Data Guard (standby replication/DR), Flashback (point-in-time recovery).
- **Strong vendor support** and mature tooling for very large, mission-critical deployments.
- **Advanced partitioning and performance tuning** capabilities for massive datasets.

13.5 Core Syntax & Examples

```
-- Table with Oracle-specific IDENTITY column (12c+)
CREATE TABLE employees (
  id NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  name VARCHAR2(100) NOT NULL,
  salary NUMBER(10,2),
  hire_date DATE DEFAULT SYSDATE
);
```

```

-- Sequence (classic approach, still widely used)
CREATE SEQUENCE emp_seq START WITH 1 INCREMENT BY 1;
INSERT INTO employees (id, name, salary) VALUES (emp_seq.NEXTVAL, 'Alice', 65000);

-- Basic SELECT with Oracle-specific ROWNUM / FETCH
SELECT name, salary FROM employees
WHERE ROWNUM <= 10
ORDER BY salary DESC;

-- Modern row-limiting (12c+)
SELECT name, salary FROM employees
ORDER BY salary DESC
FETCH FIRST 10 ROWS ONLY;

-- PL/SQL Stored Procedure
CREATE OR REPLACE PROCEDURE give_raise(
    p_emp_id IN NUMBER,
    p_percent IN NUMBER
) AS
BEGIN
    UPDATE employees
    SET salary = salary * (1 + p_percent/100)
    WHERE id = p_emp_id;

    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END give_raise;
/

-- PL/SQL block with cursor
DECLARE
    CURSOR emp_cursor IS SELECT name, salary FROM employees WHERE salary > 50000;
    v_name employees.name%TYPE;
    v_salary employees.salary%TYPE;
BEGIN
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO v_name, v_salary;
        EXIT WHEN emp_cursor%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_name || ': ' || v_salary);
    END LOOP;
    CLOSE emp_cursor;
END;
/

-- Trigger
CREATE OR REPLACE TRIGGER trg_audit_salary
AFTER UPDATE OF salary ON employees
FOR EACH ROW
BEGIN

```

```
INSERT INTO salary_audit (emp_id, old_salary, new_salary, changed_at)
VALUES (:OLD.id, :OLD.salary, :NEW.salary, SYSDATE);
END;
/
```

13.6 Basic Interview Questions

Q1. What is the difference between Oracle Database and MySQL? Oracle is a commercial, enterprise-focused RDBMS with advanced features (PL/SQL, RAC, Data Guard, fine-grained security) and licensing costs, typically used for large mission-critical enterprise systems. MySQL is open-source, simpler, lighter-weight, and dominant in web applications; Oracle generally offers more advanced enterprise tooling and scalability features out of the box.

Q2. What is PL/SQL? Oracle's **procedural extension to SQL**, adding variables, loops, conditionals, exception handling, and modular constructs (procedures, functions, packages, triggers) that execute directly inside the database engine.

Q3. What is the difference between a procedure and a function in PL/SQL? A **procedure** performs an action and may or may not return a value (via OUT parameters), called as a standalone statement. A **function** must return exactly one value (via RETURN) and can be used inline within a SQL expression (e.g., in a SELECT).

Q4. What is a sequence in Oracle? A schema object that generates a sequence of unique numeric values (via .NEXTVAL/.CURRVAL), traditionally used to generate primary key values before Oracle 12c introduced native IDENTITY columns.

Q5. What is a cursor? A pointer to a result set from a query, used in PL/SQL to iterate row-by-row over query results — explicit cursors are manually declared/opened/fetched/closed; implicit cursors are automatically managed for simple DML/single-row SELECT statements.

Q6. What is the difference between VARCHAR2 and CHAR? VARCHAR2 is variable-length (stores only the actual characters used, up to a max), more storage-efficient. CHAR is fixed-length (pads with trailing spaces to the declared length) — generally VARCHAR2 is preferred for most use cases.

13.7 Advanced Interview Questions

Q1. Explain Oracle's Multi-Version Concurrency Control (MVCC) using undo segments. Instead of read locks blocking writers (or vice versa), Oracle maintains **undo segments** recording the "before" version of changed data. When a query starts, it sees a consistent snapshot of data as of that point in time; if another transaction modifies a row afterward, readers continue to see the **old version** from undo data rather than being blocked — giving "readers don't block writers, writers don't block readers" behavior, improving concurrency significantly compared to traditional lock-based systems.

Q2. What is Oracle RAC (Real Application Clusters)? A clustering solution where **multiple database instances run on different physical servers but share the same underlying storage/database**, providing horizontal scale-out and high availability — if one node fails, others continue serving connections (failover), and workload can be load-balanced across nodes. Unlike typical master-replica replication, all RAC nodes are active and can process both reads and writes against the shared database.

Q3. Explain Oracle Data Guard. A disaster-recovery/high-availability feature that maintains one or more **standby databases** as synchronized copies of a primary database, using redo log shipping/apply. **Physical standby** applies redo directly at the block level (byte-for-byte copy); **logical standby** applies SQL-translated changes, allowing the standby to be used

for reporting queries while staying in sync. Enables fast failover/switchover in case of primary database failure or planned maintenance.

Q4. What is Oracle Flashback technology? A set of features enabling recovery from logical errors without a full restore: FLASHBACK QUERY (view/query data as of a past point in time using undo data), FLASHBACK TABLE (rewind a table to a previous state), and FLASHBACK DATABASE (rewind the entire database) — much faster than traditional point-in-time recovery from backups for correcting accidental data changes.

Q5. What is the difference between a PL/SQL package and a standalone procedure/function? A **package** groups related procedures, functions, variables, and cursors into a single named unit with a public **specification** (interface) and private **body** (implementation) — enabling encapsulation (hiding helper logic), better performance (the whole package loads into memory together, and package state can persist for a session), and easier maintenance/versioning than many scattered standalone objects.

Q6. How does Oracle's query optimizer (Cost-Based Optimizer, CBO) decide execution plans, and how do you analyze them? The CBO estimates the **cost** (I/O, CPU, memory) of many possible execution plans (join orders, join methods — nested loops/hash/merge join, index usage vs full table scan) using statistics gathered on tables/indexes (DBMS_STATS), and picks the lowest-estimated-cost plan. DBAs analyze actual plans via EXPLAIN PLAN / DBMS_XPLAN.DISPLAY and runtime execution stats (SQL Trace, AWR reports) to identify and tune poorly performing queries, often by adding indexes, updating stale statistics, or using optimizer hints.

14. MongoDB

14.1 What is it?

MongoDB is a **document-oriented NoSQL database** that stores data as flexible, JSON-like **BSON documents** (Binary JSON) grouped into **collections**, rather than rows/tables with a fixed schema. It's designed for horizontal scalability, high write throughput, and flexible/evolving data models.

14.2 When is it used?

- **Applications with flexible/evolving schemas** — content management, catalogs with varying attributes per item, user profiles with optional fields.
- **High write-throughput applications** — logging, IoT sensor data, real-time analytics.
- **Rapid prototyping** where schema is not fully finalized upfront.
- **Applications needing horizontal scalability** across many servers (sharding) for very large datasets.
- **Hierarchical/nested data** that maps naturally to a document (e.g., an order with embedded line items) rather than being spread across many joined tables.

14.3 How is it used?

1. Data is organized as **documents** (BSON, similar to JSON objects) inside **collections** (analogous to tables), inside a **database**.
2. No fixed schema is enforced by default — documents in the same collection can have different fields (though **schema validation rules** can optionally be added).
3. Query using MongoDB's query language (a JS-like syntax via the driver, or the Mongo Shell) — filter, project, sort, aggregate documents.
4. Use the **Aggregation Pipeline** for complex, multi-stage data transformations (similar to SQL's GROUP BY, but as a sequence of stages).
5. Scale horizontally via **sharding** (partitioning data across multiple servers by a shard key) and achieve high availability via **replica sets** (a primary node + secondary nodes that replicate data and can be promoted on failover).

14.4 Why is it used?

- **Schema flexibility**: easy to evolve the data model as application requirements change, without costly migrations.
- **Natural fit for hierarchical/nested data**: embedding related data in a single document avoids joins for common access patterns, improving read performance for those patterns.
- **Horizontal scalability** built-in via sharding — designed from the ground up for distributed deployments.
- **High write throughput and performance** for large volumes of semi-structured data.
- **Developer-friendly**: documents map naturally to objects/JSON in application code (especially JS/Node.js), reducing the "object-relational impedance mismatch."

14.5 Core Syntax & Examples

```
// Insert
db.employees.insertOne({
  name: "Alice",
  department: "Engineering",
```

```

    salary: 65000,
    skills: ["JavaScript", "MongoDB"],
    address: { city: "NY", zip: "10001" }
  });

db.employees.insertMany([
  { name: "Bob", department: "Sales", salary: 55000 },
  { name: "Carol", department: "Engineering", salary: 72000 }
]);

// Find / Query
db.employees.find({ department: "Engineering" });
db.employees.find({ salary: { $gt: 60000 } }).sort({ salary: -1 }).limit(10);
db.employees.find({ skills: "MongoDB" }); // array field match
db.employees.findOne({ name: "Alice" });

// Update
db.employees.updateOne(
  { name: "Alice" },
  { $set: { salary: 70000 }, $push: { skills: "React" } }
);
db.employees.updateMany({ department: "Sales" }, { $inc: { salary: 5000 } });

// Delete
db.employees.deleteOne({ name: "Bob" });

// Aggregation Pipeline
db.employees.aggregate([
  { $match: { department: "Engineering" } },
  { $group: { _id: "$department", avgSalary: { $avg: "$salary" }, count: { $sum: 1 } } },
  { $sort: { avgSalary: -1 } }
]);

// Index
db.employees.createIndex({ salary: -1 });
db.employees.createIndex({ name: 1 }, { unique: true });

```

14.6 Basic Interview Questions

Q1. What is the difference between SQL and NoSQL databases? SQL (relational) databases use structured tables with fixed schemas, relationships enforced via foreign keys, and SQL for querying — optimized for complex, multi-table transactional consistency (ACID). NoSQL databases (like MongoDB) use flexible, often schema-less data models (documents, key-value, wide-column, graph), optimized for horizontal scalability and flexible/evolving data, often trading strict consistency for availability/partition tolerance (per the CAP theorem).

Q2. What is a document and a collection in MongoDB? A **document** is a single record stored as BSON (like a JSON object) with key-value pairs, potentially including nested objects/arrays. A **collection** is a group of documents, analogous to a table in a relational database, but without enforcing a uniform schema across documents.

Q3. What is the `_id` field? Every MongoDB document has a unique `_id` field serving as its primary key; if not provided explicitly, MongoDB auto-generates a 12-byte **ObjectId** (encoding timestamp, machine identifier, process id, and counter) on insert.

Q4. What is the difference between embedding and referencing in MongoDB schema design? **Embedding** nests related data directly within a parent document (denormalization) — fast reads for data always accessed together, but can lead to large documents/duplication. **Referencing** stores a related document's `_id` and requires a separate query (or `$lookup`) to join — normalized, avoids duplication, better for data accessed independently or growing unbounded (e.g., comments on a highly popular post).

Q5. What are MongoDB indexes and why are they important? Like relational indexes, they're data structures (B-Tree by default) that speed up queries on indexed fields, avoiding full **collection scans**. Without appropriate indexes, MongoDB must scan every document to satisfy a query, which is slow at scale.

Q6. What is the Aggregation Pipeline? A framework for processing documents through a sequence of **stages** (`$match`, `$group`, `$sort`, `$project`, `$lookup`, etc.), each transforming the data and passing results to the next stage — MongoDB's equivalent of SQL's GROUP BY/JOIN/complex analytical queries.

14.7 Advanced Interview Questions

Q1. Explain MongoDB's replica set architecture and failover process. A replica set consists of a **primary** node (accepts all writes) and multiple **secondary** nodes that asynchronously replicate the primary's **oplog** (operations log). If the primary becomes unavailable, the remaining nodes hold an **election** (using a Raft-like consensus protocol) to promote a secondary to primary automatically, providing high availability without manual intervention. Read preference can be configured to route reads to secondaries for scaling read-heavy workloads (at the cost of potentially stale/eventually-consistent reads).

Q2. Explain sharding in MongoDB. Sharding horizontally partitions data across multiple servers ("shards") based on a **shard key**. The mongos query router directs operations to the appropriate shard(s) based on the shard key, while the **config servers** store cluster metadata (chunk-to-shard mapping). A well-chosen shard key (high cardinality, even distribution, aligned with query patterns) is critical — a poor shard key (e.g., a monotonically increasing field) can create "hot shards" that receive disproportionate write load.

Q3. What consistency model does MongoDB provide, and how does it relate to the CAP theorem? By default, reads from the primary are **strongly consistent** (reflect all acknowledged writes). Reads from secondaries are **eventually consistent** by default, since replication is asynchronous, though **read/write concerns** can be tuned: `writeConcern: "majority"` ensures a write is acknowledged by a majority of replica set members before returning success (durability guarantee), and `readConcern: "majority"` ensures a read only returns data acknowledged by a majority — together enabling stronger consistency guarantees at some throughput/latency cost, illustrating MongoDB's flexible position along the CAP theorem's consistency-availability spectrum.

Q4. When would you choose embedding vs referencing, in more depth? Rule of thumb: embed when data has a "**contains**" relationship, is usually accessed/updated together, and won't grow unboundedly (e.g., an address embedded in a user document). Reference when data is **shared across many parents**, grows unboundedly (e.g., millions of blog comments), needs independent querying/updating, or when strict normalization is needed to avoid update-anomaly-style duplication. MongoDB also imposes a **16MB document size limit**, which itself discourages unbounded embedding.

Q5. Explain MongoDB transactions — how do multi-document ACID transactions work? Since MongoDB 4.0, multi-document ACID transactions are supported (similar to relational transactions) using snapshot isolation, allowing multiple operations across multiple documents/collections to be committed or rolled back atomically. However, transactions

carry more performance overhead than single-document atomic operations (which were always atomic even pre-4.0), so schema design that embeds related data to keep operations single-document is still often preferred over relying heavily on multi-document transactions for performance-critical paths.

Q6. What is the WiredTiger storage engine, and what concurrency model does it use? WiredTiger (MongoDB's default storage engine since 3.2) uses **document-level concurrency control** (rather than collection- or database-level locking used by older engines), supports **compression** (reducing storage footprint and I/O), and uses **MVCC** (multi-version concurrency control) internally, giving each transaction/operation a consistent snapshot view without blocking readers against writers in most cases.

15. Git

15.1 What is it?

Git is a **distributed version control system (DVCS)** created by Linus Torvalds (2005) that tracks changes to files over time, enabling multiple developers to collaborate on the same codebase, maintain full history, branch/merge work in parallel, and revert to previous states when needed. “Distributed” means every developer’s local clone has a **complete copy of the entire repository history**, not just the latest snapshot.

15.2 When is it used?

- **Any software project**, solo or team-based, that needs history tracking and collaboration — the de facto industry standard.
- **Branching workflows** for parallel feature development, bug fixes, and release management without disrupting the main codebase.
- **Code review processes** via pull/merge requests (GitHub, GitLab, Bitbucket).
- **CI/CD pipelines**, which trigger off Git events (push, tag, merge).
- **Open-source collaboration** at scale (Linux kernel itself is Git’s original use case).

15.3 How is it used?

1. `git init` (or `git clone <url>`) to start/get a repository.
2. Make changes, `git add` to stage them, `git commit` to save a snapshot with a message.
3. `git branch/git checkout -b` to create and switch to a new branch for isolated work.
4. `git push/git pull` to sync changes with a **remote** repository (e.g., GitHub).
5. `git merge/git rebase` to integrate changes from one branch into another.
6. Resolve merge conflicts manually when Git can’t auto-merge overlapping changes.
7. Use pull requests for peer review before merging into the main branch.

Working Directory --git add--> Staging Area --git commit--> Local Repo --git push--> Remote Repo

15.4 Why is it used?

- **Full history and traceability**: every change is recorded with author, timestamp, and message — enables blame/audit and easy rollback.
- **Distributed nature**: works offline, no single point of failure, every clone is a full backup.
- **Fast branching/merging**: lightweight branches (just pointers to commits) make parallel feature development cheap and safe.
- **Industry standard**: virtually universal in professional software development, essential for team collaboration.
- **Enables CI/CD and modern DevOps workflows**.

15.5 Core Syntax & Examples

```
# Setup
git init
git clone https://github.com/user/repo.git
git config --global user.name "Alice"
git config --global user.email "alice@example.com"

# Basic workflow
```

```

git status
git add file.txt           # stage a specific file
git add .                  # stage all changes
git commit -m "Add feature X"
git push origin main
git pull origin main

# Branching
git branch feature-login   # create branch
git checkout feature-login # switch to it
git checkout -b feature-login # create + switch in one step
git switch feature-login   # modern equivalent of checkout

# Merging
git checkout main
git merge feature-login    # merge feature-login into main

# Rebasing (rewrite history for a cleaner linear log)
git checkout feature-login
git rebase main

# Viewing history
git log --oneline --graph --all
git diff                   # unstaged changes
git diff --staged          # staged changes

# Undoing changes
git restore file.txt       # discard unstaged changes to a file
git reset --soft HEAD~1   # undo last commit, keep changes staged
git reset --hard HEAD~1   # undo last commit, discard changes entirely
git revert <commit-hash>  # create a new commit that undoes a previous commit (safe for

# Stashing
git stash                  # temporarily shelve uncommitted changes
git stash pop              # reapply stashed changes

# Remote
git remote add origin https://github.com/user/repo.git
git fetch origin           # download changes without merging

```

15.6 Basic Interview Questions

Q1. What is the difference between Git and GitHub? Git is the version control tool/software itself, running locally. GitHub is a **cloud hosting service** for Git repositories, adding collaboration features (pull requests, issues, code review, CI/CD via Actions) on top of Git.

Q2. What is the difference between git pull and git fetch? git fetch downloads new commits/branches from the remote **without merging** them into your local branch (just updates your local view of remote-tracking branches). git pull is git fetch + git merge (or git rebase if configured) combined — it fetches **and** integrates the changes immediately.

Q3. What is a merge conflict, and how do you resolve one? It occurs when Git cannot automatically reconcile changes to the same lines/file made independently on two branches

being merged. Git marks the conflicting sections in the file with <<<<<<, =====, >>>>>> markers; the developer manually edits the file to the desired resolution, then `git add` the file and completes the merge with `git commit`.

Q4. What is the difference between `git merge` and `git rebase`? `git merge` combines histories by creating a new **merge commit** with two parents, preserving the true branching history. `git rebase` replays your branch's commits one-by-one on top of the target branch, producing a **linear history** without a merge commit — cleaner log, but rewrites commit hashes, which is risky on shared/public branches.

Q5. What is `.gitignore` used for? A file listing patterns for files/directories Git should **not track** (e.g., build artifacts, `node_modules/`, environment secrets, IDE config files) — keeping the repository clean of generated or sensitive files.

Q6. What is the staging area (index)? An intermediate area between the working directory and the repository where changes are collected via `git add` before being permanently recorded with `git commit` — allowing selective, granular commits rather than committing every changed file at once.

15.7 Advanced Interview Questions

Q1. Explain Git's internal object model (blobs, trees, commits). Git stores everything as content-addressed objects in `.git/objects`, identified by SHA-1 (or SHA-256 in newer configs) hashes of their content: a **blob** stores raw file content (no filename/metadata); a **tree** represents a directory, mapping filenames to blob/tree hashes (a snapshot of the file structure); a **commit** points to a single tree (the full snapshot at that point), its parent commit(s), author/committer info, and a message. This content-addressable, immutable structure is what makes Git's history integrity verifiable and enables efficient deduplication (identical file content across commits shares the same blob).

Q2. Explain `git rebase -i` (interactive rebase) and its uses. Opens an editable list of commits in a range, letting you **reorder, squash (combine), edit, drop, or reword** commits before they're replayed — commonly used to clean up messy feature-branch history (e.g., squashing "WIP" commits into one meaningful commit) before merging into a shared branch. Because it rewrites commit hashes, it should only be done on commits not yet shared/pushed to others (or coordinated carefully with a force-push if it is shared).

Q3. What is the difference between `git reset --soft`, `--mixed`, and `--hard`? All three move the current branch pointer (HEAD) to a specified commit. `--soft` leaves the working directory and staging area untouched (changes from "undone" commits remain staged). `--mixed` (default) resets the staging area but keeps working directory changes (unstaged). `--hard` resets both the staging area **and** working directory, permanently discarding any uncommitted changes — the most destructive option.

Q4. Explain the difference between `git revert` and `git reset` for undoing changes, and when to use each. `git reset` moves the branch pointer backward, effectively erasing commits from history (dangerous on shared branches since it rewrites history others may have already pulled). `git revert` creates a **new commit** that inverts the changes of a target commit, preserving full history — the safe choice for undoing changes on a branch that others are already collaborating on/have pulled.

Q5. What is a detached HEAD state, and how do you recover from it? Occurs when you check out a specific commit (or tag) directly rather than a branch — HEAD points directly to a commit instead of a branch reference. Any new commits made in this state aren't attached to any branch and can become unreachable (eventually garbage collected) once you check out something else. To keep such work, create a new branch pointing at the current commit before switching away: `git checkout -b new-branch-name`.

Q6. Explain Git's three-way merge algorithm. When merging two branches, Git identifies the **common ancestor commit** (merge base) of both branches, then compares each branch's changes against that common ancestor to determine what changed on each side. If the two branches modified different, non-overlapping parts, Git auto-merges them; if both modified the **same lines**, it flags a conflict for manual resolution, since it can't determine which change (or combination) is intended.

Q7. What are Git hooks, and give a practical example? Scripts that Git automatically executes at specific points in the workflow (e.g., pre-commit, pre-push, post-merge), stored in `.git/hooks/`. A common example: a pre-commit hook that runs a linter/formatter and blocks the commit if code style checks fail, enforcing code quality automatically before code even enters history.

16. VS Code

16.1 What is it?

Visual Studio Code (VS Code) is a free, open-source, lightweight but powerful **source code editor** developed by Microsoft, built on Electron (itself using web technologies — HTML/CSS/JS/TypeScript). It supports virtually every language and framework through a massive **extension marketplace**, built-in Git integration, an integrated terminal, and a powerful debugger.

16.2 When is it used?

- **General-purpose coding** across nearly any language (JavaScript/TypeScript, Python, Java, C++, Go, etc.) — the most widely used code editor in industry today.
- **Web and full-stack development**, given its native strength with JS/TS/Node.js tooling.
- **Remote development** — via extensions like Remote-SSH, Dev Containers, and WSL integration, editing code that lives on a remote server/container as if local.
- **Quick scripting/editing tasks** where a full heavyweight IDE (like IntelliJ/Eclipse) isn't necessary.

16.3 How is it used?

1. Open a folder as a **workspace**; VS Code indexes files for search, IntelliSense, and Git status.
2. Install **extensions** for language support, linting, formatting, debugging adapters, and productivity tools.
3. Use **IntelliSense** (context-aware code completion powered by language servers via the Language Server Protocol) for suggestions, type info, and inline documentation.
4. Use the built-in **integrated terminal** to run commands without leaving the editor.
5. Use the built-in **debugger** — set breakpoints, step through code, inspect variables — configured via `launch.json`.
6. Use built-in **Git integration** (Source Control panel) for staging, committing, branching without leaving the editor.
7. Customize behavior/appearance via `settings.json`, keybindings, and themes.

16.4 Why is it used?

- **Lightweight yet extensible**: starts fast, but the extension ecosystem lets it rival full IDEs for any language.
- **Free and open-source**, cross-platform (Windows/Mac/Linux).
- **Excellent Git integration** built in, no plugin required for basics.
- **Language Server Protocol (LSP)** support gives smart autocompletion/refactoring across many languages consistently.
- **Integrated terminal and debugger** reduce context-switching between tools.
- **Huge community and marketplace** — extensions for almost anything (Docker, Kubernetes, database clients, AI coding assistants).

16.5 Key Features & Usage Examples

```
// .vscode/launch.json - debug configuration example (Node.js)
{
  "version": "0.2.0",
  "configurations": [
```

```

{
  "type": "node",
  "request": "launch",
  "name": "Launch Program",
  "program": "${workspaceFolder}/app.js"
}
]
}

```

Common keyboard shortcuts: - Ctrl+P / Cmd+P — Quick Open (jump to any file) - Ctrl+Shift+P / Cmd+Shift+P — Command Palette (access any command) - Ctrl+\` — Toggle integrated terminal - F5 — Start debugging - Ctrl+Shift+F — Search across entire workspace - Ctrl+D — Select next occurrence of current selection (multi-cursor editing) - Ctrl+Space — Trigger IntelliSense suggestions manually

16.6 Basic Interview Questions

Q1. What is VS Code, and how is it different from a full IDE like IntelliJ? VS Code is a lightweight, highly extensible code editor with IDE-like features added via extensions (debugging, IntelliSense, Git). A full IDE like IntelliJ is a heavier, more opinionated, language-specific tool with deep built-in project/build-system understanding, advanced refactoring, and integrated tooling out of the box without needing plugins.

Q2. What is IntelliSense? VS Code's context-aware code completion feature — suggests variable/function names, shows parameter info, and inline documentation, powered by language servers (implementing the Language Server Protocol) for each language.

Q3. How do you debug code in VS Code? Set breakpoints by clicking in the gutter next to line numbers, configure a `launch.json` (or use auto-detected configurations), then press F5 to start debugging — you can step through code, inspect variable values in the Debug sidebar, and view the call stack.

Q4. What are VS Code extensions, and how do you install one? Add-ons that extend VS Code's functionality (language support, linters, themes, Git tools). Installed via the Extensions panel (Ctrl+Shift+X) or Command Palette, sourced from the Visual Studio Marketplace.

Q5. What is the integrated terminal used for? Running shell commands (npm install, git commands, build scripts) directly inside the editor without switching to a separate terminal application, keeping the developer's workflow in one window.

16.7 Advanced Interview Questions

Q1. Explain the Language Server Protocol (LSP) and why it matters architecturally. LSP is a standardized JSON-RPC-based protocol (originally developed by Microsoft for VS Code) that decouples language-specific intelligence (autocomplete, go-to-definition, diagnostics, refactoring) from the editor itself. A language server runs as a **separate process** implementing LSP for a given language; any LSP-compliant editor (VS Code, Neovim, Sublime) can talk to it. This means language tooling only needs to be built once per language, not once per editor — a major architectural win for the ecosystem.

Q2. How does VS Code enable remote development (Remote-SSH/Dev Containers), and how does it work under the hood? VS Code splits into a lightweight **UI/client** running locally and a **server component** that runs on the remote machine or inside a container, communicating over a protocol similar to LSP. Extensions, language servers, and the actual file system operations execute **on the remote side**, while only the UI renders locally — giving

a native-feeling experience while editing code that physically lives on a remote server, VM, container, or WSL instance.

Q3. What is `tasks.json`, and how does it differ from `launch.json`? `tasks.json` defines **build/automation tasks** (e.g., running a build script, linter, or test suite) invokable via the Command Palette or keybindings — VS Code doesn't manage their execution state beyond running/showing output. `launch.json` defines **debug configurations** — how to start (or attach to) a process under a debugger, enabling breakpoints, variable inspection, and step-through execution, which requires a Debug Adapter Protocol (DAP) implementation for the target language/runtime.

Q4. How does VS Code achieve good performance despite being built on Electron (a full Chromium + Node.js runtime)? Techniques include: heavy use of **web workers** and separate processes for CPU-intensive work (language servers, extensions run in a separate “Extension Host” process so a misbehaving extension can't freeze the UI thread); **virtualized rendering** of large lists/files (only rendering visible lines); lazy-loading extensions only when relevant file types are opened; and an optimized, monaco-editor-based text rendering engine (the same core editor component used to power VS Code's text buffer) built for handling very large files efficiently.

17. Eclipse

17.1 What is it?

Eclipse is a free, open-source, **full-featured Integrated Development Environment (IDE)**, historically the dominant IDE for **Java** development, built on the Eclipse Platform (itself built using OSGi and SWT/JFace for its UI). It supports many languages via plugins (C/C++ via CDT, PHP, etc.) but is most associated with enterprise Java development.

17.2 When is it used?

- **Java (especially Java EE/Jakarta EE) enterprise development** — long-standing default for many enterprise/legacy Java shops.
- **Android development** historically (before Android Studio, itself based on IntelliJ, became the official tool).
- Projects requiring specific Eclipse-based tooling — e.g., **Eclipse Modeling Framework (EMF)**, RCP (Rich Client Platform) applications built on the Eclipse Platform itself.
- Teams/organizations with existing Eclipse-based workflows, build configurations, and plugin investments.

17.3 How is it used?

1. Create/import a project (Java Project, Maven Project, Dynamic Web Project) into the **Workspace**.
2. Eclipse automatically compiles Java source incrementally in the background as you type/save (using its own built-in Java compiler, distinct from `javac`), surfacing errors immediately.
3. Use the **Package Explorer** to navigate project structure; use **Perspectives** (Java, Debug, Git) to switch between task-oriented layouts of views/editors.
4. Use built-in refactoring tools (rename, extract method, move class) that update all references project-wide.
5. Debug using the built-in debugger — breakpoints, step-through, variable/expression inspection.
6. Manage dependencies/builds via integrated **Maven (m2e)** or **Gradle (Buildship)** plugin support.
7. Extend functionality via the **Eclipse Marketplace** (plugins for frameworks, servers, version control).

17.4 Why is it used?

- **Deep, mature Java tooling:** powerful refactoring, code generation, and navigation built specifically around Java's language semantics.
- **Free and open-source**, with a long track record in enterprise environments.
- **Highly extensible plugin architecture** (OSGi-based) — can be customized into a tailored IDE for almost any workflow.
- **Integrated Application Server support** (Tomcat, WildFly, WebSphere) for deploying/debugging Java EE web applications directly from the IDE.
- **Strong legacy/enterprise adoption**, meaning many existing large Java codebases have established Eclipse-based workflows.

17.5 Key Features & Workflow Examples

Typical Java EE workflow in Eclipse: 1. File > New > Dynamic Web Project → configure target runtime (e.g., Apache Tomcat). 2. Write servlets/JSPs/Spring controllers in the project. 3. Right-click project → Run As > Run on Server — Eclipse deploys and starts the app on the configured server, opening a browser/internal view. 4. Set breakpoints and use Debug As > Debug on Server to attach the debugger to the running server process.

Common shortcuts: - Ctrl+Shift+T — Open Type (jump to any class by name) - Ctrl+Shift+R — Open Resource (jump to any file) - Ctrl+1 — Quick Fix (suggests fixes for errors/warnings) - Alt+Shift+R — Rename refactor - Ctrl+Space — Content Assist (autocomplete)

17.6 Basic Interview Questions

Q1. What is Eclipse primarily used for? It's an IDE most commonly associated with Java and Java EE enterprise development, offering project management, code editing with real-time compilation, debugging, and integrated build/deployment tooling.

Q2. What is a Workspace vs a Project in Eclipse? A **Workspace** is a top-level directory holding metadata and settings for a group of related **Projects**; each Project is an individual codebase (e.g., one Java application) with its own source folders, build path, and configuration.

Q3. How does Eclipse handle Java compilation differently from running javac manually? Eclipse uses its own built-in incremental **Java compiler (ECJ - Eclipse Compiler for Java)** that recompiles only changed files in the background as you save, immediately surfacing errors/warnings in the editor — unlike manually running javac, which requires an explicit, full compile step.

Q4. What is the Package Explorer? The primary navigation view in Eclipse's Java perspective, showing the project's package/class structure in a tree, letting developers navigate source files, resources, and libraries.

Q5. How do you manage project dependencies in Eclipse? Via the **Build Path** configuration (for manually added JARs), or more commonly today via **Maven** (using the m2e plugin, reading pom.xml) or **Gradle** (via Buildship), which automatically resolve and download dependencies.

17.7 Advanced Interview Questions

Q1. Explain Eclipse's plugin architecture (OSGi) and why it matters. Eclipse itself is built as a set of modular, dynamically loadable/unloadable **OSGi bundles** (plugins) — even core IDE features (Java tooling, the UI framework itself) are plugins. This architecture allows Eclipse to be extended or even repurposed into an entirely different application (like an Eclipse RCP desktop app) by adding/removing bundles, with each bundle declaring explicit dependencies and versioned interfaces — enabling strong modularity and avoiding "JAR hell" classloading conflicts between plugins.

Q2. What is the difference between the Eclipse Compiler for Java (ECJ) and javac, and why does Eclipse use its own? ECJ is Eclipse's independent, standards-compliant Java compiler implementation, decoupled from Oracle's javac. Eclipse uses it because it needs to support **incremental compilation** integrated tightly with its editor (compiling on every keystroke/save for instant error feedback) and needs to be able to compile code that has errors elsewhere in the project (so you can still run/debug a class even if an unrelated file has a compile error) — capabilities not well suited to invoking the standard javac tool directly.

Q3. Explain Eclipse’s refactoring engine — how does “Rename” safely update all references project-wide? Refactoring operations parse and build an in-memory **Abstract Syntax Tree (AST)** representation of the relevant source files (via the Java Development Tools’ AST/binding resolution), performing full semantic analysis to distinguish, for example, a local variable named `x` from an unrelated field also named `x` in another class. This lets “Rename” (or “Extract Method”, “Move Class”, etc.) safely update every genuine reference across the entire project while leaving unrelated same-named symbols untouched — a capability that simple text find-and-replace cannot provide safely.

Q4. What is Eclipse RCP (Rich Client Platform), and how does it relate to the base Eclipse IDE? Eclipse RCP is the underlying framework (SWT/JFace UI toolkit + OSGi module system + the workbench/perspective/view model) that the Eclipse IDE itself is built on top of. Developers can use the same framework to build entirely custom, standalone desktop applications unrelated to software development (e.g., specialized enterprise desktop tools) — the Eclipse IDE is essentially “just” one particular RCP application built from these reusable components.

18. IntelliJ IDEA

18.1 What is it?

IntelliJ IDEA is a powerful, intelligent **IDE for JVM languages** (Java, Kotlin, Scala, Groovy) developed by JetBrains, widely regarded for its deep static-analysis-driven code intelligence, superior refactoring tools, and out-of-the-box productivity features. It comes in a free **Community Edition** (Java/Kotlin, core features) and a paid **Ultimate Edition** (adds full-stack web/enterprise framework support: Spring, Java EE, database tools, JS/TS).

18.2 When is it used?

- **Modern Java and Kotlin development** — the de facto standard IDE for Android development (Android Studio is built directly on IntelliJ's platform) and increasingly preferred over Eclipse for general Java work.
- **Spring/Spring Boot enterprise application development** — Ultimate Edition has deep, first-class Spring tooling (bean navigation, application context visualization).
- **Full-stack projects** where a single IDE needs to handle back-end (Java/Kotlin) and front-end (JS/TS/HTML/CSS) code together (Ultimate Edition).
- Projects requiring advanced code analysis, inspections, and safe automated refactoring at scale.

18.3 How is it used?

1. Open/import a project — IntelliJ auto-detects build systems (Maven pom.xml, Gradle build.gradle) and configures the project structure automatically.
2. IntelliJ continuously performs **static analysis in the background**, powering real-time code inspections (highlighting bugs, code smells, style issues) even before compiling.
3. Use deep **IntelliSense-style autocomplete** that understands type context, project structure, and even framework-specific conventions (e.g., suggesting Spring bean names).
4. Use powerful **refactoring tools** (rename, extract method/variable/interface, change signature, inline) that safely propagate changes project-wide.
5. Use the integrated **debugger, built-in terminal, Git/VCS integration**, and **database tools** (Ultimate) — all without leaving the IDE.
6. Run/debug directly via **Run Configurations** for applications, JUnit tests, or Spring Boot apps.

18.4 Why is it used?

- **Best-in-class code intelligence**: deep semantic understanding of code (not just text-based suggestions), catching bugs and suggesting fixes before you even run the code.
- **Superior, safe refactoring** — widely considered the gold standard among IDEs for large-scale automated refactors.
- **Excellent out-of-the-box experience**: minimal plugin configuration needed for most JVM workflows, unlike Eclipse which often needs plugin setup.
- **First-class Kotlin support** (JetBrains created Kotlin), important given Kotlin's growing role in Android and back-end development.
- **Integrated tooling** (DB clients, HTTP client, Docker, VCS) reduces context switching.

18.5 Key Features & Examples

Common shortcuts (Windows/Linux — Mac uses Cmd instead of Ctrl): - Shift+Shift (double Shift) — “Search Everywhere” (files, classes, actions, symbols) - Ctrl+Alt+L — Reformat code - Alt+Enter — Show intention actions / quick fixes (e.g., auto-import, generate getter/setter) - Ctrl+Shift+F10 / right-click Run — Run current file/test - Shift+F6 — Rename refactor (updates all references) - Ctrl+Alt+M — Extract Method refactor

Live Templates example: typing `sout` + Tab auto-expands to `System.out.println();` — a built-in code snippet expansion shortcut.

18.6 Basic Interview Questions

Q1. What is IntelliJ IDEA primarily used for, and what languages does it support? Primarily Java and Kotlin development (its core strength), also supporting Groovy, Scala, and — in the Ultimate Edition — full-stack web technologies (JavaScript/TypeScript, HTML/CSS), Spring, and various enterprise frameworks.

Q2. What is the difference between IntelliJ Community and Ultimate editions? Community is free and open-source, covering core Java/Kotlin/Android development. Ultimate is a paid edition adding support for web/enterprise frameworks (Spring, Java EE, Micronaut), full JS/TS/front-end tooling, database tools, and application server integration.

Q3. How does IntelliJ handle project/build systems like Maven and Gradle? It has built-in support that auto-detects `pom.xml/build.gradle`, automatically syncing the project structure, dependencies, and source roots based on the build file — no manual project configuration needed as with older Eclipse workflows.

Q4. What are “Live Templates” in IntelliJ? Predefined (and customizable) code snippet shortcuts — typing an abbreviation (like `psvm` for public static void main) and pressing Tab expands it into full boilerplate code, speeding up repetitive typing.

Q5. What is “Search Everywhere” / double-Shift used for? A universal search feature to quickly find and jump to any file, class, symbol, or IDE action/setting by typing a few characters — a core productivity shortcut for navigating large codebases without using the mouse.

18.7 Advanced Interview Questions

Q1. How does IntelliJ’s code inspection engine work, and why is it considered more powerful than simple linting? IntelliJ builds a full **semantic model (PSI — Program Structure Interface, plus resolved type/reference bindings)** of the code as you type, not just tokenizing text like a simple linter. This lets it run hundreds of built-in “inspections” that understand actual program semantics — e.g., detecting a genuinely unreachable code path, a resource that’s opened but never closed, or a call that will throw `NullPointerException` given the surrounding null-checks — and offer context-aware quick fixes (Alt+Enter), rather than just flagging pattern-based style violations.

Q2. Explain how IntelliJ’s refactoring (e.g., “Change Signature”) safely updates an entire large codebase. Refactorings operate on the resolved semantic model, not raw text: when you invoke “Change Signature” on a method (adding/removing/reordering parameters), IntelliJ identifies **every actual call site** referencing that specific method (correctly distinguishing overloaded methods with the same name but different signatures), then rewrites each call site to match the new signature — including handling default values for newly added parameters — all before you confirm the change, often with a preview step showing exactly what will change.

Q3. What is the IntelliJ Platform, and how does it relate to Android Studio, PyCharm, WebStorm, etc.? The **IntelliJ Platform** is JetBrains' shared underlying IDE framework (project model, UI framework, VCS integration, plugin system) that all their IDEs are built on top of, each configured/bundled with a different set of language plugins: PyCharm = Platform + Python plugins, WebStorm = Platform + JS/TS plugins, Android Studio = Platform + Android/Kotlin plugins (maintained by Google in partnership with JetBrains). This is why the UI, keyboard shortcuts, and many core behaviors (Search Everywhere, refactoring, VCS panel) feel consistent across all of them.

Q4. How does IntelliJ's indexing process work, and why does it matter for performance/IntelliSense accuracy? On opening/updating a project, IntelliJ builds and maintains **indexes** (stub trees / cached symbol tables for classes, methods, fields across the project and all its dependencies, including compiled library JARs) rather than re-parsing files on every keystroke. This upfront indexing (visible as the "Indexing" progress bar) is what enables instantaneous "Find Usages," accurate autocomplete across the entire dependency graph, and fast navigation (Ctrl+Click to definition) even in very large, multi-module projects — at the cost of an initial indexing time and disk space for the index cache.